

Machine Learning - Python Pandas

July 8, 2026

1 PAO25_25_07_Python_Pandas



Autor: Elvis Pachacama **Recopilado por:** Pablo Aguilar * Repositorio: GitHub

1.1 PAO 25-25 - Pandas

1.2 1. Pandas

- Librería (de facto estándar) para estructurar datos tabulares.
- Multivariable (string, int, float, bool...).
- Dos clases:
 - Series (1 dimensión)
 - DataFrames (2+ dimensiones)

```
[4]: # librería externa
import pandas as pd
from pandas import Series, DataFrame

# Línea 1: comentario descriptivo.
# Línea 2: importa la librería pandas con el alias estándar 'pd'.
# Línea 3: importa directamente las clases Series y DataFrame para poder
↪usarlas sin el
# prefijo pd. (por ejemplo Series(...) en lugar de pd.Series(...)).
# Resultado esperado: no hay salida visible; quedan disponibles pd, Series y
↪DataFrame.
```

```
[5]: pd.__version__

# Línea 1: accede al atributo __version__ del módulo pandas, que contiene la
↳ versión instalada
#           como string; al ser la última expresión de la celda, Jupyter la
↳ muestra como salida.
# Resultado esperado: un string con la versión instalada, por ejemplo '2.2.2'
↳ (el valor exacto
#           depende del entorno).
```

```
[5]: '3.0.3'
```

1.3 2. Series

- Datos unidimensionales (similar a NumPy).
- Elementos + índices modificables.

```
[6]: countries = pd.Series(['Spain', 'Andorra', 'Gibraltar', 'Portugal', 'France'])
print(countries)

# Línea 1: pd.Series crea una estructura unidimensional a partir de una lista
↳ de strings; al
#           no especificar índice, pandas asigna automáticamente un índice
↳ numérico entero
#           empezando en 0.
# Línea 2: imprime la Serie (muestra índice, valor y el dtype al final).
# Resultado esperado:
# 0      Spain
# 1    Andorra
# 2  Gibraltar
# 3    Portugal
# 4      France
# dtype: object
```

```
0      Spain
1    Andorra
2  Gibraltar
3    Portugal
4      France
dtype: str
```

```
[7]: # especificando el índice
countries = pd.Series(['Spain', 'Andorra', 'Gibraltar', 'Portugal', 'France'],
                      index=range(10,60,10))
print(countries)

# Línea 1: comentario descriptivo.
```

```
# Línea 2-3: crea la misma Serie de países, pero especificando explícitamente
↳ el parámetro
#          index=range(10,60,10), que genera los valores 10,20,30,40,50 como
↳ etiquetas de índice.
# Línea 4: imprime la Serie con el índice personalizado.
# Resultado esperado:
# 10      Spain
# 20      Andorra
# 30      Gibraltar
# 40      Portugal
# 50      France
# dtype: object
```

```
10      Spain
20      Andorra
30      Gibraltar
40      Portugal
50      France
dtype: str
```

```
[8]: # los índices pueden ser de más tipos
football_cities = pd.Series(['Barcelona', 'Madrid', 'Valencia', 'Sevilla'],
                             index=['a', 'b', 'c', 'd'])
print(football_cities)

# Línea 1: comentario descriptivo.
# Línea 2-3: crea una Serie con etiquetas de índice tipo string
↳ ('a', 'b', 'c', 'd') en lugar de
#          enteros, demostrando que el índice de una Series acepta cualquier
↳ tipo hashable.
# Línea 4: imprime la Serie.
# Resultado esperado:
# a      Barcelona
# b      Madrid
# c      Valencia
# d      Sevilla
# dtype: object
```

```
a      Barcelona
b      Madrid
c      Valencia
d      Sevilla
dtype: str
```

```
[9]: # Atributos
football_cities.name = 'Ciudades con dos equipos en primera' # nombrar la Serie
football_cities.index.name = 'Id' # Describir los índices
print(football_cities)
```

```

# Línea 1: comentario descriptivo.
# Línea 2: asigna un nombre descriptivo a la Serie completa (atributo .name),
    ↳ útil como
#           cabecera de columna si luego se convierte en DataFrame.
# Línea 3: asigna un nombre al índice (.index.name), que aparece como cabecera
    ↳ de la columna
#           de índices al imprimir.
# Línea 4: imprime la Serie ya con nombre e índice nombrado.
# Resultado esperado:
# Id
# a      Barcelona
# b      Madrid
# c      Valencia
# d      Sevilla
# Name: Ciudades con dos equipos en primera, dtype: object

```

```

Id
a      Barcelona
b      Madrid
c      Valencia
d      Sevilla
Name: Ciudades con dos equipos en primera, dtype: str

```

```

[10]: # acceso similar a NumPy o listas, según posición (depreacted)
print(football_cities[2])
# acceso a través del índice semántico
print(football_cities['c'])
print(football_cities['c'] == football_cities[2])

# Línea 1: comentario descriptivo.
# Línea 2: accede por posición entera (2) igual que en una lista/ndarray; en
    ↳ versiones recientes
#           de pandas esta forma está en desuso (deprecated) para Series con
    ↳ índice no entero.
# Línea 3: comentario descriptivo.
# Línea 4: accede usando el índice semántico 'c' (la etiqueta), forma
    ↳ recomendada.
# Línea 5: compara que ambos accesos devuelven el mismo valor.
# Resultado esperado:
# Valencia
# Valencia
# True

```

KeyError

Traceback (most recent call last)

```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\indexes\base.py:3641, in Index.get_loc(self, key)
    3640 try:
-> 3641     return self._engine.get_loc(casted_key)
    3642 except KeyError as err:

```

```

File pandas/_libs/index.pyx:168, in pandas._libs.index.IndexEngine.get_loc()
--> 168 'Could not get source, probably due dynamically evaluated source code.'

```

```

File pandas/_libs/index.pyx:176, in pandas._libs.index.IndexEngine.get_loc()
--> 176 'Could not get source, probably due dynamically evaluated source code.'

```

```

File pandas/_libs/index.pyx:583, in pandas._libs.index.StringObjectEngine._check_type()
--> 583 'Could not get source, probably due dynamically evaluated source code.'

```

KeyError: 2

The above exception was the direct cause of the following exception:

KeyError Traceback (most recent call last)

Cell In[10], line 2

```

1 # acceso similar a NumPy o listas, según posición (depeccrated)
----> 2 print(football_cities[2])
3 # acceso a través del índice semántico
4 print(football_cities['c'])
5 print(football_cities['c'] == football_cities[2])

```

```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\series.py:
-> 959, in Series._getitem__(self, key)
    954     key = unpack_1tuple(key)
    956 elif key_is_scalar:
    957     # Note: GH#50617 in 3.0 we changed int key to always be treated as
    958     # a label, matching DataFrame behavior.
-> 959     return self._get_value(key)
    961 # Convert generator to list before going through hashable part
    962 # (We will iterate through the generator there to check for slices)
    963 if is_iterator(key):

```

```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\series.py:
-> 1046, in Series._get_value(self, label, takeable)
    1043     return self._values[label]
    1045 # Similar to Index.get_value, but we do not fall back to positional
-> 1046 loc = self.index.get_loc(label)
    1048 if is_integer(loc):
    1049     return self._values[loc]

```

```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\indexes\base.py:3648, in Index.get_loc(self, key)
    3643     if isinstance(casted_key, slice) or (
    3644         isinstance(casted_key, abc.Iterable)
    3645         and any(isinstance(x, slice) for x in casted_key)
    3646     ):
    3647         raise InvalidIndexError(key) from err
-> 3648     raise KeyError(key) from err
    3649 except TypeError:
    3650     # If we have a listlike key, _check_indexing_error will raise
    3651     # InvalidIndexError. Otherwise we fall through and re-raise
    3652     # the TypeError.
    3653     self._check_indexing_error(key)

KeyError: 2

```

1.4 3. Tratamiento similar a ndarray

```

[11]: # múltiple recolección de elementos
print(football_cities[ ['a','c'] ])
print(football_cities[ [0, 3] ])

# Línea 1: comentario descriptivo.
# Línea 2: al pasar una lista de etiquetas ['a','c'] se seleccionan varios
    ↪ elementos a la vez
#           (fancy indexing) devolviendo una nueva Serie con esas dos ciudades.
# Línea 3: análogamente, con una lista de posiciones [0,3] se seleccionan la
    ↪ primera y la cuarta
#           fila por posición.
# Resultado esperado:
# Id
# a    Barcelona
# c    Valencia
# Name: ..., dtype: object
# Id
# a    Barcelona
# d    Sevilla
# Name: ..., dtype: object

```

```

Id
a    Barcelona
c    Valencia
Name: Ciudades con dos equipos en primera, dtype: str

```

```

-----
KeyError                                Traceback (most recent call last)
Cell In[11], line 3

```

```

1 # múltiple recolección de elementos
2 print(football_cities[ ['a','c'] ])
----> 3 print(football_cities[ [0, 3] ])
4
5 # Línea 1: comentario descriptivo.
6 # Línea 2: al pasar una lista de etiquetas ['a','c'] se seleccionan
↳ varios elementos a la vez

```

```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\series.py:
↳991, in Series._getitem__(self, key)
    988     key = np.asarray(key, dtype=bool)
    989     return self._get_rows_with_mask(key)
-> 991 return self._get_with(key)

```

```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\series.py:
↳1003, in Series._get_with(self, key)
    1000 elif isinstance(key, tuple):
    1001     return self._get_values_tuple(key)
-> 1003 return self.loc[key]

```

```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\indexing.py
↳1207, in _LocationIndexer._getitem__(self, key)
    1205 maybe_callable = com.apply_if_callable(key, self.obj)
    1206 maybe_callable = self._raise_callable_usage(key, maybe_callable)
-> 1207 return self._getitem_axis(maybe_callable, axis=axis)

```

```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\indexing.py
↳1438, in _LocIndexer._getitem_axis(self, key, axis)
    1435     if hasattr(key, "ndim") and key.ndim > 1:
    1436         raise ValueError("Cannot index with multidimensional key")
-> 1438 return self._getitem_iterable(key, axis=axis)
    1440 # nested tuple slicing
    1441 if is_nested_tuple(key, labels):

```

```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\indexing.py
↳1378, in _LocIndexer._getitem_iterable(self, key, axis)
    1375 self._validate_key(key, axis)
    1377 # A collection of keys
-> 1378 keyarr, indexer = self._get_listlike_indexer(key, axis)
    1379 return self.obj.reindex_with_indexers(
    1380     {axis: [keyarr, indexer]}, allow_dups=True
    1381 )

```

```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\indexing.py
↳1576, in _LocIndexer._get_listlike_indexer(self, key, axis)
    1573 ax = self.obj._get_axis(axis)
    1574 axis_name = self.obj._get_axis_name(axis)
-> 1576 keyarr, indexer = ax._get_indexer_strict(key, axis_name)

```

```
1578 return keyarr, indexer
```

```
File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\indexes\base.py:6302, in Index._get_indexer_strict(self, key, axis_name)
```

```
→ py:6302, in Index._get_indexer_strict(self, key, axis_name)
6299 else:
6300     keyarr, indexer, new_indexer = self._reindex_non_unique(keyarr)
→ 6302 self._raise_if_missing(keyarr, indexer, axis_name)
6304 keyarr = self.take(indexer)
6305 if isinstance(key, Index):
6306     # GH 42790 - Preserve name from an Index
```

```
File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\indexes\base.py:6352, in Index._raise_if_missing(self, key, indexer, axis_name)
```

```
→ py:6352, in Index._raise_if_missing(self, key, indexer, axis_name)
6350 if nmissing:
6351     if nmissing == len(indexer):
→ 6352         raise KeyError(f"None of [{key}] are in the [{axis_name}]")
6354     not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].
→ unique())
6355     raise KeyError(f"{not_found} not in index")
```

```
KeyError: "None of [Index([0, 3], dtype='int64', name='Id')] are in the [index]"
```

```
[12]: # slicing
print(football_cities[:'c']) # incluye ambos extremos con el indice semantico
print(football_cities[:2])

# Línea 1: comentario descriptivo.
# Línea 2: al hacer slicing con etiquetas semánticas (':c'), pandas incluye
→ AMBOS extremos
# (a diferencia del slicing posicional), es decir, selecciona desde
→ 'a' hasta 'c' inclusive.
# Línea 3: el slicing posicional [:2] funciona como en listas/ndarrays: excluye
→ el índice 2,
# devolviendo las dos primeras filas (posiciones 0 y 1).
# Resultado esperado:
# Id
# a    Barcelona
# b      Madrid
# c    Valencia
# Name: ..., dtype: object
# Id
# a    Barcelona
# b      Madrid
# Name: ..., dtype: object
```

Id

a Barcelona


```

b      Madrid
c      Valencia
Name: Ciudades con dos equipos en primera, dtype: str
Id
a      Barcelona
b      Madrid
Name: Ciudades con dos equipos en primera, dtype: str

```

```

[13]: #cast a list
lista = list(football_cities[:'c'])
print(lista)
print(type(lista))

# Línea 1: comentario descriptivo.
# Línea 2: list() convierte los VALORES de la Serie (el slice hasta 'c') en una
↳ lista Python
#          normal (se pierden los índices).
# Línea 3: imprime la lista.
# Línea 4: imprime el tipo, confirmando que ahora es una lista pura de Python.
# Resultado esperado:
# ['Barcelona', 'Madrid', 'Valencia']
# <class 'list'>

```

```

['Barcelona', 'Madrid', 'Valencia']
<class 'list'>

```

```

[14]: type(football_cities[:'c'])

# Línea 1: obtiene el tipo del objeto devuelto por el slice; al ser la última
↳ expresión de la
#          celda se muestra automáticamente.
# Resultado esperado: pandas.core.series.Series (el slice sigue siendo una
↳ Series, no una lista).

```

```

[14]: pandas.Series

```

```

[15]: #cast a ndarray
import numpy as np
cities = np.array(football_cities[:'c'])
print(cities)
print(type(cities))

# Línea 1: comentario descriptivo.
# Línea 2: importa numpy.
# Línea 3: np.array convierte los valores de la Serie en un ndarray de NumPy
↳ (también se
#          pierden los índices).
# Línea 4: imprime el ndarray.

```

```
# Línea 5: imprime el tipo, confirmando que ahora es un numpy.ndarray.
# Resultado esperado:
# ['Barcelona' 'Madrid' 'Valencia']
# <class 'numpy.ndarray'>
```

```
['Barcelona' 'Madrid' 'Valencia']
<class 'numpy.ndarray'>
```

```
[16]: # cas a dictionary
lista = dict(football_cities[:'c'])
print(lista)

# Línea 1: comentario descriptivo (cast a diccionario).
# Línea 2: dict() convierte la Serie en un diccionario Python usando las
    ↪ etiquetas de índice
#         como claves y los valores de la Serie como valores del diccionario.
# Línea 3: imprime el diccionario resultante.
# Resultado esperado: {'a': 'Barcelona', 'b': 'Madrid', 'c': 'Valencia'}
```

```
{'a': 'Barcelona', 'b': 'Madrid', 'c': 'Valencia'}
```

```
[17]: # uso de masks para seleccionar
fibonacci = pd.Series([0, 1, 1, 2, 3, 5, 8, 13, 21])
print(fibonacci)
mask = fibonacci > 10
print(mask)
print(fibonacci[mask])

dst = pd.Series([13,21])
print(dst)
dst.equals(fibonacci)

fb = fibonacci[mask]
fb.reset_index(drop=True, inplace=True)
print(fb)
dst.equals(fb)

# Línea 1: comentario descriptivo.
# Línea 2: crea una Serie con los primeros números de Fibonacci (índice
    ↪ automático 0..8).
# Línea 3: la imprime.
# Línea 4: crea una máscara booleana: True donde el valor es mayor a 10.
# Línea 5: imprime la máscara.
# Línea 6: usa la máscara para filtrar solo los valores de Fibonacci mayores a
    ↪ 10 (conservando
#         sus índices originales 7 y 8).
# Línea 7: crea otra Serie 'dst' con valores [13,21] e índice automático 0,1.
# Línea 8: la imprime.
```

```

# Línea 9: dst.equals(fibonacci) compara ambas Series completas (valores E
↳ ÍNDICES); al tener
#
tamaños e índices distintos, el resultado es False (esta línea no
↳ imprime nada por
#
sí sola en un script, pero en Jupyter se muestra si es la última
↳ línea de la celda;
#
aquí hay más código después, por lo que su valor se descarta).
# Línea 10: filtra de nuevo fibonacci con la máscara, guardando en 'fb'
↳ (índices 7 y 8, valores 13,21).
# Línea 11: reset_index(drop=True, inplace=True) reindexa 'fb' para que use
↳ índices 0,1
#
consecutivos, descartando (drop=True) los índices originales 7,8, y
↳ modifica 'fb' in place.
# Línea 12: imprime 'fb' ya con índice reiniciado.
# Línea 13: dst.equals(fb) ahora compara dos Series con los mismos valores
↳ [13,21] y el mismo
#
índice [0,1]; al ser la última expresión de la celda, Jupyter
↳ muestra su resultado.
# Resultado esperado: se imprime la Serie fibonacci completa, la máscara
↳ booleana, los valores
#
filtrados con índices 7 y 8, la Serie dst, y finalmente 'fb' con
↳ índice 0,1 y valores
#
13,21; el resultado final mostrado por Jupyter es True (dst y fb ya
↳ coinciden).

```

```

0      0
1      1
2      1
3      2
4      3
5      5
6      8
7     13
8     21
dtype: int64
0      False
1      False
2      False
3      False
4      False
5      False
6      False
7       True
8       True
dtype: bool
7     13
8     21

```

```
dtype: int64
0      13
1      21
dtype: int64
0      13
1      21
dtype: int64
```

```
[17]: True
```

```
[18]: # aplicar funciones de numpy a la serie
import numpy as np
print(np.sum(fibonacci))

# Línea 1: comentario descriptivo.
# Línea 2: importa numpy (por si no estaba ya importado en el kernel).
# Línea 3: np.sum funciona directamente sobre una Series de pandas (que
↳ internamente usa un
#      ndarray) y suma todos sus valores.
# Resultado esperado: 54 (0+1+1+2+3+5+8+13+21).
```

54

```
[19]: #filtrado con np.where
distances = pd.Series([12.1,np.nan,12.8,76.9,6.1,7.2])
valid_distances = np.where(pd.notnull(distances),distances,2)
print(valid_distances)
print(type(valid_distances))

# Línea 1: comentario descriptivo.
# Línea 2: crea una Serie de distancias que incluye un valor NaN.
# Línea 3: pd.notnull(distances) genera una máscara booleana True donde el
↳ valor NO es nulo;
#      np.where usa esa máscara para conservar el valor original donde es
↳ válido y sustituir
#      por 2 donde es NaN.
# Línea 4: imprime el resultado.
# Línea 5: imprime el tipo del resultado.
# Resultado esperado:
# [12.1  2.  12.8 76.9  6.1  7.2]
# <class 'numpy.ndarray'>
```

```
[12.1  2.  12.8 76.9  6.1  7.2]
<class 'numpy.ndarray'>
```

1.4.1 3.0.1 Iteración

```
[20]: # iterar sobre elementos
for value in fibonacci:
    print('Value: ' + str(value))

# # iterar sobre índices
for index in fibonacci.index:
    print('Index: ' + str(index))

# Línea 1: comentario descriptivo.
# Línea 2: al iterar directamente sobre una Series con un for, se recorren solo
↳ sus VALORES
#           (no los índices), igual que al iterar una lista.
# Línea 3: imprime cada valor con un prefijo descriptivo.
# Línea 4: comentario descriptivo (doblemente comentado en el original, se
↳ mantiene como texto).
# Línea 5: fibonacci.index expone el objeto Index; iterar sobre él recorre solo
↳ las ETIQUETAS
#           de índice.
# Línea 6: imprime cada índice con un prefijo descriptivo.
# Resultado esperado: se imprimen 9 líneas "Value: ..." con los valores de
↳ Fibonacci en orden,
#           seguidas de 9 líneas "Index: ..." con los índices 0 al 8.
```

```
Value: 0
Value: 1
Value: 1
Value: 2
Value: 3
Value: 5
Value: 8
Value: 13
Value: 21
Index: 0
Index: 1
Index: 2
Index: 3
Index: 4
Index: 5
Index: 6
Index: 7
Index: 8
```

```
[21]: # iterar sobre elementos e índices al mismo tiempo
for index, value in fibonacci.items():
    print('Index: ' + str(index) + ' Value: ' + str(value))
```

```

# Línea 1: comentario descriptivo.
# Línea 2: .items() devuelve pares (índice, valor), permitiendo
↳ desempaquetarlos directamente
#
# en la variable del for, similar a .items() en un diccionario.
# Línea 3: imprime cada par índice-valor en una sola línea.
# Resultado esperado: 9 líneas del tipo 'Index: 0 Value: 0', 'Index: 1 Value:
↳ 1', ...,
#
# 'Index: 8 Value: 21'.

```

```

Index: 0 Value: 0
Index: 1 Value: 1
Index: 2 Value: 1
Index: 3 Value: 2
Index: 4 Value: 3
Index: 5 Value: 5
Index: 6 Value: 8
Index: 7 Value: 13
Index: 8 Value: 21

```

```

[22]: for index, value in zip(fibonacci.index, fibonacci):
        print('Index: ' + str(index) + ' Value: ' + str(value))

# Línea 1: zip combina en paralelo el iterable de índices (fibonacci.index) con
↳ el iterable
#
# de valores (fibonacci), produciendo el mismo efecto que .items()
↳ pero de forma manual.
# Línea 2: imprime cada par índice-valor.
# Resultado esperado: idéntico al de la celda anterior, 9 líneas 'Index: N
↳ Value: V'.

```

```

Index: 0 Value: 0
Index: 1 Value: 1
Index: 2 Value: 1
Index: 3 Value: 2
Index: 4 Value: 3
Index: 5 Value: 5
Index: 6 Value: 8
Index: 7 Value: 13
Index: 8 Value: 21

```

1.4.2 3.1 Series como diccionarios

- Interpretar el índice como clave.
- Acepta operaciones para diccionarios.

```

[23]: # crear una serie a partir de un diccionario
serie = pd.Series( { 'Carlos' : 100, 'Marcos': 98} )
print(serie.index)

```

```

print(serie.values)
print(serie)
print(type(serie))

# Línea 1: comentario descriptivo.
# Línea 2: pd.Series a partir de un diccionario usa automáticamente las CLAVES
↳ del diccionario
#           como índice y los VALORES del diccionario como los datos de la Serie.
# Línea 3: imprime el objeto Index (las claves 'Carlos', 'Marcos').
# Línea 4: imprime un ndarray con los valores (100, 98).
# Línea 5: imprime la Serie completa (índice + valores).
# Línea 6: imprime el tipo del objeto.
# Resultado esperado:
# Index(['Carlos', 'Marcos'], dtype='object')
# [100  98]
# Carlos    100
# Marcos     98
# dtype: int64
# <class 'pandas.core.series.Series'>

```

```

Index(['Carlos', 'Marcos'], dtype='str')
[100  98]
Carlos    100
Marcos     98
dtype: int64
<class 'pandas.Series'>

```

```

[24]: # añade y elimina elementos a través de índices
serie['Pedro'] = 12
serie['Pedro']=15
del serie['Marcos']
print(serie)

# Línea 1: comentario descriptivo.
# Línea 2: asignar a una clave nueva ('Pedro') AÑADE un nuevo elemento a la
↳ Serie, igual que en
#           un diccionario.
# Línea 3: volver a asignar a la misma clave ACTUALIZA su valor (de 12 a 15),
↳ no crea un duplicado.
# Línea 4: del elimina el elemento con clave 'Marcos', igual que en un
↳ diccionario.
# Línea 5: imprime la Serie resultante.
# Resultado esperado:
# Carlos    100
# Pedro     15
# dtype: int64

```

```

Carlos    100

```

Pedro 15
dtype: int64

```
[25]: # query una serie
# print(serie['Marcos'])
if 'Marcos' in serie:
    print(serie['Marcos'])
print(serie)

# Línea 1: comentario descriptivo.
# Línea 2: (comentada intencionalmente) acceder directamente a serie['Marcos']
    ↳ lanzaría un
#         KeyError porque 'Marcos' ya fue eliminado en la celda anterior; se
    ↳ deja comentada
#         para no interrumpir la ejecución.
# Línea 3: el operador 'in' sobre una Series (al igual que en un diccionario)
    ↳ comprueba si la
#         clave/índice existe, sin lanzar error.
# Línea 4: como 'Marcos' ya no está, este print nunca se ejecuta.
# Línea 5: imprime la Serie completa (sin 'Marcos').
# Resultado esperado: no se imprime nada relacionado con 'Marcos' (la condición
    ↳ es False); solo
#         se imprime la Serie con Carlos y Pedro.
```

Carlos 100
Pedro 15
dtype: int64

1.4.3 3.2 Operaciones entre series

```
[26]: # suma de dos series
# suma de valores con el mismo índice (NaN si no aparece en ambas)
serie1 = pd.Series([10,20,30,40], index=range(4) )
serie2 = pd.Series([1,2,3], index=range(3) )
suma = serie1 + serie2
print(suma)

# Línea 1-2: comentarios descriptivos.
# Línea 3: crea serie1 con 4 valores e índice 0..3.
# Línea 4: crea serie2 con 3 valores e índice 0..2.
# Línea 5: al sumar dos Series, pandas ALINEA por índice: suma los valores que
    ↳ comparten el
#         mismo índice, y donde un índice solo existe en una de las dos Series
    ↳ el resultado es
#         NaN (aquí, el índice 3 solo está en serie1).
# Línea 6: imprime el resultado de la suma.
# Resultado esperado:
```



```
# 0    11.0
# 1    22.0
# 2    33.0
# 3     NaN
# dtype: float64
```

```
0    11.0
1    22.0
2    33.0
3     NaN
dtype: float64
```

```
[27]: # resta de series (similar a la suma)
print(serie1 - serie2)

# Línea 1: comentario descriptivo.
# Línea 2: igual que la suma, la resta se alinea por índice; el índice 3 (solo
    ↪ en serie1) da NaN.
# Resultado esperado:
# 0     9.0
# 1    18.0
# 2    27.0
# 3     NaN
# dtype: float64
```

```
0     9.0
1    18.0
2    27.0
3     NaN
dtype: float64
```

```
[28]: # operaciones de pre-filtrado
result = serie1 + serie2
result[pd.isnull(result)] = 0 # mask con isnull()
print(result)

# Línea 1: comentario descriptivo.
# Línea 2: repite la suma alineada por índice (con NaN en la posición 3).
# Línea 3: pd.isnull(result) genera una máscara True donde el valor es NaN; se
    ↪ usa esa máscara
#           para reemplazar esos NaN por 0 directamente en 'result'
    ↪ (modificación in place).
# Línea 4: imprime el resultado ya sin valores NaN.
# Resultado esperado:
# 0    11.0
# 1    22.0
# 2    33.0
# 3     0.0
```

```
# dtype: float64

0    11.0
1    22.0
2    33.0
3     0.0
dtype: float64
```

3.2.1 Diferencias entre Pandas Series y diccionario

- Diccionario: estructura que relaciona claves y valores de forma arbitraria.
- Series: estructura estricta de valores con una lista de índices asignados por posición.
- Series es más eficiente para ciertas operaciones que los diccionarios.
- En las Series los valores de entrada pueden ser listas o NumPy arrays.
- En Series los índices semánticos pueden ser enteros o caracteres, igual que los valores.
- Series podría entenderse como algo intermedio entre una lista y un diccionario Python, pero de una dimensión.

1.5 4. DataFrame

- Datos tabulares (filas x columnas).
- Columnas: Series con índices compartidos.

```
[29]: # crear un DataFrame a partir de un diccionario de elementos de la misma longitud
      ↪ longitud
diccionario = { "Nombre" : ["Marisa","Laura","Manuel"],
                "Edad"   : [34,29,12] }

print(diccionario)
# las claves identifican columnas
frame = pd.DataFrame(diccionario)
display(frame)

# Línea 1: comentario descriptivo.
# Línea 2-3: define un diccionario donde cada clave será una columna y cada
      ↪ lista asociada
#           contiene los valores de esa columna (ambas listas deben tener la
      ↪ misma longitud).
# Línea 4: imprime el diccionario original en formato Python.
# Línea 5: comentario explicando que las claves del diccionario se vuelven
      ↪ columnas.
# Línea 6: pd.DataFrame construye la tabla a partir del diccionario.
# Línea 7: display() (función de Jupyter) renderiza el DataFrame como una tabla
      ↪ HTML con formato.
# Resultado esperado: el diccionario impreso como texto, y luego una tabla de 3
      ↪ filas x 2
#           columnas (Nombre, Edad) con índice automático 0,1,2.
```

```
{'Nombre': ['Marisa', 'Laura', 'Manuel'], 'Edad': [34, 29, 12]}
```

	Nombre	Edad
0	Marisa	34
1	Laura	29
2	Manuel	12

```
[30]: # crear un DataFrame a partir de un diccionario de elementos de la misma longitud
diccionario = { "Nombre" : ["Marisa","Laura","Manuel"],
                "Edad" : [34,29,12] }
# las claves identifican columnas
frame = pd.DataFrame(diccionario, index = ['a', 'b', 'c'])
display(frame)

# Línea 1: comentario descriptivo.
# Línea 2-3: se redefine el mismo diccionario.
# Línea 4: comentario descriptivo.
# Línea 5: se construye el DataFrame indicando explícitamente el parámetro index para usar
# etiquetas 'a','b','c' en lugar del índice numérico por defecto.
# Línea 6: muestra la tabla resultante.
# Resultado esperado: tabla de 3 filas (índices a, b, c) x 2 columnas (Nombre, Edad).
```

	Nombre	Edad
a	Marisa	34
b	Laura	29
c	Manuel	12

```
[31]: # además de 'index', el parámetro 'columns' especifica el número y orden de las columnas
frame = pd.DataFrame(diccionario, columns = ['Nacionalidad', 'Nombre', 'Edad', 'Profesion', 'Genero'])
display(frame)

# Línea 1: comentario descriptivo.
# Línea 2: el parámetro 'columns' fuerza el DataFrame a tener exactamente esas 5 columnas en ese orden; las columnas que existen en el diccionario ('Nombre','Edad') se rellenan con sus datos, y las que no existen ('Nacionalidad','Profesion','Genero') se crean vacías con valores NaN.
# Línea 3: muestra la tabla resultante.
# Resultado esperado: tabla de 3 filas x 5 columnas; 'Nacionalidad', 'Profesion' y 'Genero' aparecen completamente con NaN, mientras 'Nombre' y 'Edad' tienen los datos originales.
```

	Nacionalidad	Nombre	Edad	Profesion	Genero
0	NaN	Marisa	34	NaN	NaN
1	NaN	Laura	29	NaN	NaN
2	NaN	Manuel	12	NaN	NaN

```
[32]: # acceso a columnas
nombres = frame['Nombre']
display(nombres)
print(type(nombres))

edades = frame['Edad']
display(edades)
print(type(edades))

# Línea 1: comentario descriptivo.
# Línea 2: frame['Nombre'] extrae la columna 'Nombre' como una Series
#           independiente (comparte
#           el índice del DataFrame).
# Línea 3: la muestra.
# Línea 4: confirma que el tipo es pandas.core.series.Series.
# Línea 5-8: repite el mismo proceso para la columna 'Edad'.
# Resultado esperado: se muestran las Series 'Nombre' (con NaN si aplica según
#           el frame previo)
#           y 'Edad', cada una seguida de la confirmación de tipo Series.
```

```
0    Marisa
1    Laura
2    Manuel
Name: Nombre, dtype: str

<class 'pandas.Series'>

0    34
1    29
2    12
Name: Edad, dtype: int64

<class 'pandas.Series'>
```

```
[33]: #siempre que el nombre de la columna lo permita (espacios, ...)
nombres = frame.Profesion
display(nombres)
type(nombres)

# Línea 1: comentario descriptivo.
# Línea 2: además de frame['col'], se puede acceder a una columna como atributo
#           del objeto
#           (frame.Profesion), siempre que el nombre de la columna sea un
#           identificador Python
```

```
#          válido (sin espacios ni caracteres especiales).
# Línea 3: muestra la columna 'Profesion'.
# Línea 4: al ser la última expresión de la celda, se muestra el tipo del
↳ resultado.
# Resultado esperado: la columna 'Profesion' (llena de NaN según el frame de la
↳ celda anterior)
#          y el tipo pandas.core.series.Series.
```

```
0    NaN
1    NaN
2    NaN
Name: Profesion, dtype: object
```

[33]: pandas.Series

```
[34]: # acceso al primer nombre del DataFrame frame??
print(frame['Nombre'][0])
print(frame.Nombre[0])
print(nombres[0])

# Línea 1: comentario descriptivo (pregunta retórica del material).
# Línea 2: accede a la columna 'Nombre' y luego, dentro de esa Series, al
↳ elemento de índice 0.
# Línea 3: forma equivalente usando notación de atributo (frame.Nombre) en
↳ lugar de corchetes.
# Línea 4: 'nombres' en este punto sigue siendo la Series 'Profesion' de la
↳ celda anterior (por
#          eso esta línea accede al primer valor de PROFESION, no de Nombre,
↳ algo a tener en
#          cuenta ya que la variable fue reasignada previamente).
# Resultado esperado: 'Marisa' impreso dos veces, y luego el primer valor de la
↳ columna
#          'Profesion' (NaN, ya que esa columna no tenía datos).
```

```
Marisa
Marisa
nan
```

1.5.1 4.0.1 Formas de crear un DataFrame

- Con una Serie de pandas.
- Lista de diccionarios.
- Diccionario de Series de Pandas.
- Con un array de NumPy de dos dimensiones.
- Con array estructurado de NumPy.

1.5.2 4.1 Modificar DataFrames

```
[35]: # añadir columnas
diccionario = { "Nombre" : ["Marisa","Laura","Manuel"],
                "Edad" : [34,29,12] }
frame = pd.DataFrame(diccionario, columns=['Nacionalidad', 'Nombre',
↳ 'Edad', 'Profesion', 'Direccion'])
frame['Direccion'] = 'Desconocida'
display(frame)

# Línea 1: comentario descriptivo.
# Línea 2-3: redefine el diccionario base.
# Línea 4: crea el DataFrame con columnas explícitas (algunas vacías por no
↳ estar en el diccionario).
# Línea 5: asignar un único valor escalar ('Desconocida') a una columna
↳ completa hace
#          broadcasting: rellena TODAS las filas de esa columna con ese mismo
↳ valor.
# Línea 6: muestra el resultado.
# Resultado esperado: tabla de 3 filas x 5 columnas donde 'Direccion' tiene
↳ 'Desconocida' en
#          las tres filas, y 'Nacionalidad' y 'Profesion' siguen en NaN.
```

	Nacionalidad	Nombre	Edad	Profesion	Direccion
0	NaN	Marisa	34	NaN	Desconocida
1	NaN	Laura	29	NaN	Desconocida
2	NaN	Manuel	12	NaN	Desconocida

```
[36]: lista_direcciones = ['Rue 13 del Percebe, 13', 'Evergreen Terrace, 3', 'Av de
↳ los Rombos, 12']

# Línea 1: se define una lista de 3 direcciones (una por cada fila del
↳ DataFrame).
# Resultado esperado: no hay salida impresa; la lista queda almacenada en la
↳ variable
#          'lista_direcciones' para usarse en la siguiente celda.
```

```
[37]: frame['Direccion'] = lista_direcciones
display(frame)

# Línea 1: asignar una lista (del mismo largo que el número de filas) a una
↳ columna reemplaza
#          el valor de cada fila con el elemento correspondiente de la lista,
↳ en orden.
# Línea 2: muestra el DataFrame actualizado.
# Resultado esperado: la columna 'Direccion' ahora contiene las tres
↳ direcciones específicas en
```

```
#          lugar de 'Desconocida'.
```

	Nacionalidad	Nombre	Edad	Profesion	Direccion
0	NaN	Marisa	34	NaN	Rue 13 del Percebe, 13
1	NaN	Laura	29	NaN	Evergreen Terrace, 3
2	NaN	Manuel	12	NaN	Av de los Rombos, 12

```
[38]: # añadir fila (requiere todos los valores)
user_2 = ['Alemania', 'Klaus', 20, 'none', 'Desconocida']
frame.loc[3] = user_2
display(frame)

# Línea 1: comentario descriptivo.
# Línea 2: se define una lista con un valor para cada una de las 5 columnas del
↳ DataFrame, en
#          el mismo orden en que aparecen las columnas.
# Línea 3: frame.loc[3] = user_2 añade una NUEVA fila con etiqueta de índice 3,
↳ asignando cada
#          valor de la lista a la columna correspondiente en ese mismo orden.
# Línea 4: muestra el DataFrame con la nueva fila añadida.
# Resultado esperado: tabla de 4 filas; la nueva fila (índice 3) contiene los
↳ datos de Klaus.
```

	Nacionalidad	Nombre	Edad	Profesion	Direccion
0	NaN	Marisa	34	NaN	Rue 13 del Percebe, 13
1	NaN	Laura	29	NaN	Evergreen Terrace, 3
2	NaN	Manuel	12	NaN	Av de los Rombos, 12
3	Alemania	Klaus	20	none	Desconocida

```
[39]: # eliminar fila (similar a Series)
frame = pd.DataFrame(diccionario, columns=['Nacionalidad', 'Nombre', 'Edad',
↳ 'Profesion'])
frame = frame.drop(2) # por qué necesitamos reasignar el frame?
display(frame)
frame.drop('Nombre', axis = 1, inplace = True)
display(frame)

# Línea 1: comentario descriptivo.
# Línea 2: recrea el DataFrame original de 3 filas x 4 columnas a partir del
↳ diccionario base.
# Línea 3: drop(2) elimina la fila de índice 2 y devuelve un NUEVO DataFrame
↳ (no modifica el
#          original in place por defecto), por lo que hay que reasignarlo a
↳ 'frame' para
#          conservar el cambio (de ahí la pregunta retórica del comentario).
# Línea 4: muestra el DataFrame ya sin la fila 2.
# Línea 5: drop('Nombre', axis=1, inplace=True) elimina la COLUMNA 'Nombre'
↳ (axis=1 = columnas);
```

```
# inplace=True modifica 'frame' directamente sin necesidad de
↳reasignar.
# Línea 6: muestra el DataFrame final sin la columna 'Nombre'.
# Resultado esperado: primero una tabla de 2 filas (índices 0 y 1) x 4
↳columnas, luego la misma
#
    tabla pero sin la columna 'Nombre' (2 filas x 3 columnas).
```

	Nacionalidad	Nombre	Edad	Profesion
0	NaN	Marisa	34	NaN
1	NaN	Laura	29	NaN

	Nacionalidad	Edad	Profesion
0	NaN	34	NaN
1	NaN	29	NaN

```
[40]: #eliminar columna
del frame['Profesion']
display(frame)

# Línea 1: comentario descriptivo.
# Línea 2: del sobre frame['Profesion'] elimina esa columna directamente del
↳DataFrame (in
#
    place), de forma similar a eliminar una clave de un diccionario.
# Línea 3: muestra el DataFrame resultante.
# Resultado esperado: tabla de 2 filas x 2 columnas (Nacionalidad, Edad).
```

	Nacionalidad	Edad
0	NaN	34
1	NaN	29

```
[41]: # acceder a la traspuesta (como una matriz)
display(frame.T)

# Línea 1: comentario descriptivo.
# Línea 2: el atributo .T devuelve la matriz transpuesta del DataFrame: las
↳columnas pasan a
#
    ser filas y viceversa (útil para inspeccionar tablas con muchas
↳columnas).
# Resultado esperado: una tabla donde las filas ahora son 'Nacionalidad' y
↳'Edad', y las
#
    columnas son los índices originales (0 y 1).
```

	0	1
Nacionalidad	NaN	NaN
Edad	34	29

1.5.3 4.2 Iteración

```
[42]: # iteración sobre el DataFrame?
frame = pd.DataFrame(diccionario, columns=['Nacionalidad', 'Nombre', 'Edad', 'Profesion'])
display(frame)
for a in frame:
    print(a) # qué es 'a'?
    print(type(a))

# Línea 1: comentario descriptivo (pregunta retórica).
# Línea 2: recrea el DataFrame completo de 3 filas x 4 columnas.
# Línea 3: lo muestra.
# Línea 4: al iterar directamente sobre un DataFrame con un for, NO se recorren
↳ las filas sino
# los NOMBRES DE LAS COLUMNAS (a diferencia de lo que podría esperarse
↳ por analogía
# con listas/arrays).
# Línea 5: imprime cada nombre de columna ('a' es un string con el nombre).
# Línea 6: imprime el tipo de 'a' (str), confirmando que se está iterando sobre
↳ las columnas.
# Resultado esperado: la tabla mostrada, y luego 4 pares de líneas del tipo
# Nacionalidad
# <class 'str'>
# (uno por cada columna: Nacionalidad, Nombre, Edad, Profesion).
```

	Nacionalidad	Nombre	Edad	Profesion
0	NaN	Marisa	34	NaN
1	NaN	Laura	29	NaN
2	NaN	Manuel	12	NaN

```
Nacionalidad
<class 'str'>
Nombre
<class 'str'>
Edad
<class 'str'>
Profesion
<class 'str'>
```

```
[43]: # iteracion sobre filas
for value in frame.values:
    print(value)
    print(type(value))

# Línea 1: comentario descriptivo.
# Línea 2: frame.values convierte el DataFrame en un ndarray 2D; iterar sobre
↳ él recorre cada
```

```
#          FILA como un array 1D.
# Línea 3: imprime cada fila (un ndarray con los valores de esa fila).
# Línea 4: imprime el tipo de cada fila (numpy.ndarray).
# Resultado esperado: 3 pares de líneas, una por cada fila del DataFrame,
↳mostrando el array de
#          esa fila y el tipo numpy.ndarray.
```

```
[nan 'Marisa' 34 nan]
<class 'numpy.ndarray'>
[nan 'Laura' 29 nan]
<class 'numpy.ndarray'>
[nan 'Manuel' 12 nan]
<class 'numpy.ndarray'>
```

```
[44]: # iterar sobre filas y luego sobre cada valor?
for values in frame.values:
    for value in values:
        print(value)

# Línea 1: comentario descriptivo (pregunta retórica).
# Línea 2: itera sobre cada fila (array 1D) del DataFrame, igual que antes.
# Línea 3: dentro de cada fila, un segundo for itera sobre cada valor
↳individual de esa fila.
# Línea 4: imprime cada valor por separado.
# Resultado esperado: se imprime, uno por línea, cada celda individual del
↳DataFrame recorrida
#          fila por fila (NaN, 'Marisa', 34, NaN, NaN, 'Laura', 29, NaN, ...).
```

```
nan
Marisa
34
nan
nan
Laura
29
nan
nan
Manuel
12
nan
```

1.5.4 4.3 Indexación y slicing con DataFrames

```
[45]: d1 = {'ciudad':'Valencia', 'temperatura':10, 'o2':1}
d2 = {'ciudad':'Barcelona', 'temperatura':8}
d3 = {'ciudad':'Valencia', 'temperatura':9}
d4 = {'ciudad':'Madrid', 'temperatura':10, 'humedad':80}
d5 = {'ciudad':'Sevilla', 'temperatura':15, 'humedad':50, 'co2':6}
```

```

d6 = {'ciudad': 'Valencia', 'temperatura': 10, 'humedad': 90, 'co2': 10}
ls_data = [d1, d2, d3, d4, d5, d6] # lista de diccionarios
df_data = pd.DataFrame(ls_data, index = list('abcdef'))
display(df_data)

# Línea 1-6: se definen 6 diccionarios individuales, cada uno representando una
↳ fila; nótese
#           que NO todos tienen las mismas claves (algunos tienen 'o2', otros
↳ 'humedad', otros
#           'co2'), simulando datos heterogéneos o incompletos.
# Línea 7: se agrupan todos los diccionarios en una lista (una de las formas de
↳ crear un
#           DataFrame: lista de diccionarios).
# Línea 8: pd.DataFrame a partir de una lista de diccionarios crea
↳ automáticamente una columna
#           por cada clave distinta encontrada en cualquiera de los diccionarios;
↳ donde una fila
#           no tenga esa clave, el valor será NaN. Se asignan índices 'a' a 'f'.
# Línea 9: muestra la tabla resultante.
# Resultado esperado: tabla de 6 filas x 5 columnas (ciudad, temperatura, o2,
↳ humedad, co2),
#           con NaN en las combinaciones donde el diccionario original no tenía
↳ ese dato.

```

	ciudad	temperatura	o2	humedad	co2
a	Valencia	10	1.0	NaN	NaN
b	Barcelona	8	NaN	NaN	NaN
c	Valencia	9	NaN	NaN	NaN
d	Madrid	10	NaN	80.0	NaN
e	Sevilla	15	NaN	50.0	6.0
f	Valencia	10	NaN	90.0	10.0

```

[46]: # Acceso a un valor concreto por índice posicional [row, col]
print(df_data.iloc[1,1])
# Acceso a todos los valores hasta un índice por enteros
display(df_data.iloc[:3,:4])
# Acceso a datos de manera explícita, índice semántico (se incluyen)
display(df_data.loc['a', 'temperatura'])
display(df_data.loc[:, 'o2'])
display(df_data.loc[:, 'temperatura':'o2'])
display(df_data.loc[:, ['ciudad', 'o2']])

# Línea 1: comentario descriptivo.
# Línea 2: .iloc[1,1] accede por posición entera: fila 1 ('b'), columna 1
↳ (temperatura).
# Línea 3: comentario descriptivo.

```

```

# Línea 4: .iloc[:3,:4] selecciona las primeras 3 filas (posiciones 0,1,2) y
↳ las primeras 4
#
#     columnas (posiciones 0..3), excluyendo el límite superior como en
↳ slicing normal.
# Línea 5: comentario descriptivo.
# Línea 6: .loc['a','temperatura'] accede por etiquetas semánticas: fila 'a',
↳ columna 'temperatura'.
# Línea 7: .loc['c', : 'o2'] selecciona filas desde el inicio hasta 'c'
↳ INCLUSIVE, y columnas
#
#     desde el inicio hasta 'o2' INCLUSIVE (el slicing por etiquetas
↳ incluye ambos extremos).
# Línea 8: .loc['c', 'temperatura': 'o2'] selecciona filas hasta 'c' inclusive,
↳ y solo las
#
#     columnas entre 'temperatura' y 'o2' inclusive.
# Línea 9: .loc[:, ['ciudad','o2']] selecciona todas las filas pero solo las
↳ columnas 'ciudad' y
#
#     'o2' (lista explícita de columnas).
# Resultado esperado: 8.0 (temperatura de la fila 'b'); una tabla 3x4; el valor
↳ 10 (temperatura
#
#     de 'a'); tablas parciales según los rangos de filas/columnas
↳ descritos, y finalmente
#
#     una tabla con solo 'ciudad' y 'o2' para las 6 filas.

```

8

	ciudad	temperatura	o2	humedad
a	Valencia	10	1.0	NaN
b	Barcelona	8	NaN	NaN
c	Valencia	9	NaN	NaN

np.int64(10)

	ciudad	temperatura	o2
a	Valencia	10	1.0
b	Barcelona	8	NaN
c	Valencia	9	NaN

	temperatura	o2
a	10	1.0
b	8	NaN
c	9	NaN

	ciudad	o2
a	Valencia	1.0
b	Barcelona	NaN
c	Valencia	NaN
d	Madrid	NaN
e	Sevilla	NaN
f	Valencia	NaN

```
[47]: # indexación con nombre de columna (por columnas)
print(df_data['ciudad']) # --> Series
display(df_data[['ciudad', 'o2']])

# Línea 1: comentario descriptivo.
# Línea 2: indexar el DataFrame con un solo nombre de columna entre corchetes
    ↳ devuelve esa
#         columna como una Series.
# Línea 3: indexar con una LISTA de nombres de columnas devuelve un DataFrame
    ↳ con solo esas
#         columnas (aunque sea una sola columna en la lista, seguiría siendo
    ↳ un DataFrame).
# Resultado esperado: la columna 'ciudad' impresa como Series, seguida de una
    ↳ tabla con las
#         columnas 'ciudad' y 'o2' para las 6 filas.
```

```
a    Valencia
b    Barcelona
c    Valencia
d      Madrid
e      Sevilla
f    Valencia
Name: ciudad, dtype: str
```

```
      ciudad  o2
a  Valencia  1.0
b  Barcelona NaN
c  Valencia  NaN
d   Madrid   NaN
e   Sevilla  NaN
f  Valencia  NaN
```

```
[48]: # indexación con índice posicional (no permitido!). Esto busca columna.
df_data[0]

# Línea 1: comentario descriptivo advirtiéndolo del comportamiento.
# Línea 2: df_data[0] NO se interpreta como 'fila en posición 0'; pandas
    ↳ intenta buscar una
#         COLUMNA cuyo nombre sea 0, y como no existe ninguna columna llamada
    ↳ 0, esto lanza un
#         KeyError.
# Resultado esperado: KeyError: 0 (excepción, no una tabla).
```

```
-----
KeyError                                Traceback (most recent call last)
File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\indexes\base.py:
    ↳ 3641, in Index.get_loc(self, key)
    3640 try:
```

```

-> 3641     return self._engine.get_loc(casted_key)
    3642 except KeyError as err:

File pandas/_libs/index.pyx:168, in pandas._libs.index.IndexEngine.get_loc()
--> 168 'Could not get source, probably due dynamically evaluated source code.'

File pandas/_libs/index.pyx:176, in pandas._libs.index.IndexEngine.get_loc()
--> 176 'Could not get source, probably due dynamically evaluated source code.'

File pandas/_libs/index.pyx:583, in pandas._libs.index.StringObjectEngine.
    ↪_check_type()
--> 583 'Could not get source, probably due dynamically evaluated source code.'

KeyError: 0

```

The above exception was the direct cause of the following exception:

```

KeyError                                Traceback (most recent call last)
Cell In[48], line 2
      1 # indexación con índice posicional (no permitido!). Esto busca columna.
----> 2 df_data[0]
      3
      4 # Línea 1: comentario descriptivo advirtiéndolo del comportamiento.
      5 # Línea 2: df_data[0] NO se interpreta como 'fila en posición 0'; pandas
    ↪ intenta buscar una

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\frame.py:
    ↪4378, in DataFrame.__getitem__(self, key)
    4374
    4375         if is_single_key:
    4376             if self.columns.nlevels > 1:
    4377                 return self._getitem_multilevel(key)
-> 4378             indexer = self.columns.get_loc(key)
    4379             if is_integer(indexer):
    4380                 indexer = [indexer]
    4381         else:

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\indexes\base.py:
    ↪3648, in Index.get_loc(self, key)
    3643     if isinstance(casted_key, slice) or (
    3644         isinstance(casted_key, abc.Iterable)
    3645         and any(isinstance(x, slice) for x in casted_key)
    3646     ):
    3647         raise InvalidIndexError(key) from err
-> 3648     raise KeyError(key) from err
    3649 except TypeError:
    3650     # If we have a listlike key, _check_indexing_error will raise
    3651     # InvalidIndexError. Otherwise we fall through and re-raise

```

```

3652     # the TypeError.
3653     self._check_indexing_error(key)

```

KeyError: 0

```

[49]: # indexar por posición con 'iloc'
print(df_data.iloc[0]) # --> Series de la primera fila (qué marca los índices)

# Línea 1: comentario descriptivo.
# Línea 2: .iloc[0] accede a la fila en la posición 0 (la primera),
    ↪devolviéndola como una
#           Series donde el índice de esa Series pasa a ser el NOMBRE DE LAS
    ↪COLUMNAS originales.
# Resultado esperado: una Series con 'ciudad'-'>'Valencia', 'temperatura'-'>10,
    ↪'o2'-'>1.0, y NaN
#           para 'humedad' y 'co2'.

```

```

ciudad      Valencia
temperatura      10
o2              1.0
humedad         NaN
co2             NaN
Name: a, dtype: object

```

```

[50]: # indexar semántico con 'loc'
df_data.loc['a'] # --> Series de la fila con índice 'a'

# Línea 1: comentario descriptivo.
# Línea 2: .loc['a'] accede a la fila con etiqueta 'a' (equivale a la primera
    ↪fila en este
#           caso), devolviéndola como Series.
# Resultado esperado: idéntico al resultado de iloc[0] en este caso, ya que 'a'
    ↪es la primera fila.

```

```

[50]: ciudad      Valencia
temperatura      10
o2              1.0
humedad         NaN
co2             NaN
Name: a, dtype: object

```

```

[51]: # indexar semántico con 'loc'
df_data.loc[:'b'] # --> DataFrame de la fila con índice 'a' y 'b'

# Línea 1: comentario descriptivo.
# Línea 2: .loc[:'b'] selecciona todas las filas desde el inicio hasta la
    ↪etiqueta 'b'

```

```
#          INCLUSIVE, devolviendo un DataFrame (no una Series) porque son
↪varias filas.
# Resultado esperado: tabla con las filas 'a' y 'b'.
```

```
[51]:      ciudad  temperatura  o2  humedad  co2
a   Valencia      10  1.0      NaN  NaN
b   Barcelona      8  NaN      NaN  NaN
```

```
[52]: df_data.loc[:, 'b'].loc[:, ["o2", "humedad"]] # slicing anidado

# Línea 1: primero se seleccionan las filas 'a' y 'b' con .loc[:, 'b'], y sobre
↪ese resultado
#          intermedio se aplica un segundo .loc para quedarse solo con las
↪columnas 'o2' y
#          'humedad' (slicing/indexación encadenada).
# Resultado esperado: tabla de 2 filas ('a', 'b') x 2 columnas ('o2', 'humedad').
```

```
[52]:      o2  humedad
a   1.0      NaN
b   NaN      NaN
```

```
[53]: # si se modifica una porcion del dataframe se modifica el dataframe original
↪(referencia)
display(df_data)
serie = df_data.loc['a']
print(serie)
serie.iloc[2] = 3000 # setting with copy warning!!!
display(df_data)

# copiar data frame
df_2 = df_data.loc['a'].copy()
df_2.iloc[2] = 3000
display(df_2)
display(df_data)

# Línea 1: comentario descriptivo.
# Línea 2: muestra el DataFrame original completo.
# Línea 3: df_data.loc['a'] devuelve una Series que, según los casos, puede ser
↪una VISTA sobre
#          los datos originales.
# Línea 4: imprime esa Series.
# Línea 5: modifica el tercer elemento (posición 2, columna 'o2') de la Series
↪'serie'; pandas
#          suele emitir un SettingWithCopyWarning porque no siempre es seguro
↪saber si se está
#          modificando una vista o una copia.
```



```
# Línea 6: muestra df_data de nuevo: en muchos casos esta modificación SÍ se
↳refleja en el
# DataFrame original (comportamiento de referencia, no garantizado y
↳por eso el warning).
# Línea 7: comentario descriptivo sobre cómo evitar el problema.
# Línea 8: se crea df_2 usando .copy() explícitamente, garantizando
↳independencia de memoria.
# Línea 9: se modifica el mismo elemento en la copia.
# Línea 10: muestra df_2 modificado.
# Línea 11: muestra df_data, que esta vez permanece SIN cambios porque df_2 es
↳una copia real.
# Resultado esperado: se observa cómo la primera modificación puede alterar el
↳DataFrame
# original (con warning de por medio), mientras que al usar .copy()
↳explícitamente el
# DataFrame original queda protegido de cambios posteriores.
```

	ciudad	temperatura	o2	humedad	co2
a	Valencia	10	1.0	NaN	NaN
b	Barcelona	8	NaN	NaN	NaN
c	Valencia	9	NaN	NaN	NaN
d	Madrid	10	NaN	80.0	NaN
e	Sevilla	15	NaN	50.0	6.0
f	Valencia	10	NaN	90.0	10.0

```
ciudad      Valencia
temperatura      10
o2              1.0
humedad         NaN
co2             NaN
Name: a, dtype: object
```

	ciudad	temperatura	o2	humedad	co2
a	Valencia	10	1.0	NaN	NaN
b	Barcelona	8	NaN	NaN	NaN
c	Valencia	9	NaN	NaN	NaN
d	Madrid	10	NaN	80.0	NaN
e	Sevilla	15	NaN	50.0	6.0
f	Valencia	10	NaN	90.0	10.0

```
ciudad      Valencia
temperatura      10
o2             3000
humedad         NaN
co2             NaN
Name: a, dtype: object
```

	ciudad	temperatura	o2	humedad	co2
a	Valencia	10	1.0	NaN	NaN

b	Barcelona	8	NaN	NaN	NaN
c	Valencia	9	NaN	NaN	NaN
d	Madrid	10	NaN	80.0	NaN
e	Sevilla	15	NaN	50.0	6.0
f	Valencia	10	NaN	90.0	10.0

```
[54]: # ambos aceptan 'axis' como argumento
# df_data.iloc(axis=1)[0] # --> todos los valores asignados a la primera
#       ↪ columna 'ciudad'
df_data.loc(axis=1)['ciudad'] # --> equivalente frame['ciudad']

# Línea 1: comentario descriptivo.
# Línea 2: (comentada) forma alternativa de usar iloc especificando axis=1 para
#       ↪ referirse a
#       columnas en vez de filas; se deja comentada como referencia.
# Línea 3: .loc(axis=1)['ciudad'] es equivalente a df_data['ciudad'], usando
#       ↪ axis=1 para indicar
#       explícitamente que se está indexando por columna.
# Resultado esperado: la columna 'ciudad' como Series, igual que
#       ↪ df_data['ciudad'].
```

```
[54]: a    Valencia
b    Barcelona
c    Valencia
d    Madrid
e    Sevilla
f    Valencia
Name: ciudad, dtype: str
```

```
[55]: # qué problema puede tener este fragmento?
frame = pd.DataFrame({"Name" : ['Carlos', 'Pedro'], "Age" : [34,22]},
#       ↪ index=[1,0])
display(frame)

# Línea 1: comentario descriptivo (pregunta retórica).
# Línea 2: crea un DataFrame de 2 filas con un índice EXPLÍCITO pero invertido:
#       ↪ la fila 0
#       (Carlos) tiene etiqueta de índice 1, y la fila 1 (Pedro) tiene
#       ↪ etiqueta de índice 0.
#       Esto genera ambigüedad, ya que el índice ya no coincide con la
#       ↪ posición.
# Línea 3: muestra la tabla.
# Resultado esperado: una tabla donde la primera fila mostrada tiene índice 1
#       ↪ (Carlos) y la
#       segunda índice 0 (Pedro); el problema es que frame[0] o frame.loc[0]/
#       ↪ frame.iloc[0]
```

```
# ya NO se refieren a la misma fila, lo cual puede generar errores de
↳ interpretación.
```

```
      Name  Age
1  Carlos   34
0  Pedro    22
```

```
[56]: # por defecto, pandas interpreta índice posicional --> error en frames
# cuando hay posible ambigüedad, utilizar loc y iloc
print('Primera fila\n')
print(frame.iloc[0])

print('\nElemento con index 0\n')
print(frame.loc[0])

# Línea 1-2: comentarios explicando la recomendación de usar loc/iloc cuando
↳ hay ambigüedad.
# Línea 3: imprime un encabezado de texto.
# Línea 4: .iloc[0] accede por POSICIÓN a la primera fila física (Carlos,
↳ aunque su etiqueta de
# índice sea 1).
# Línea 5: imprime otro encabezado de texto.
# Línea 6: .loc[0] accede por ETIQUETA al elemento cuyo índice es 0 (Pedro,
↳ aunque esté en la
# segunda posición física), demostrando la diferencia clave entre
↳ posición y etiqueta.
# Resultado esperado:
# Primera fila
# Name      Carlos
# Age         34
# Name: 1, dtype: object
#
# Elemento con index 0
# Name      Pedro
# Age         22
# Name: 0, dtype: object
```

Primera fila

```
Name      Carlos
Age         34
Name: 1, dtype: object
```

Elemento con index 0

```
Name      Pedro
Age         22
Name: 0, dtype: object
```

1.5.5 4.4 Objeto Index de Pandas

```
[57]: # Contrucción de índices
ind = pd.Index([2, 3, 5, 23, 26])
# recuperar datos
print(ind[3])
print(ind[:2])

# Línea 1: comentario descriptivo.
# Línea 2: pd.Index crea explícitamente un objeto Index independiente, que
    ↪ luego puede
#           reutilizarse para varias Series o DataFrames.
# Línea 3: comentario descriptivo.
# Línea 4: accede por posición al elemento de índice 3 dentro del objeto Index
    ↪ (valor 23).
# Línea 5: ind[:2] hace slicing tomando de 2 en 2 elementos (posiciones 0,2,4).
# Resultado esperado:
# 23
# Index([2, 5, 26], dtype='int64')
```

23

Index([2, 5, 26], dtype='int64')

```
[58]: # usar un objeto index al crear dataframe
frame = pd.DataFrame({"Name" : ['Carlos', 'Pedro', 'Manolo', 'Luis', 'Alberto'],
    ↪ "Age" : [34,22,15,55,23]}, index=ind)
display(frame)

# Línea 1: comentario descriptivo.
# Línea 2: se crea un DataFrame de 5 filas, reutilizando el objeto Index 'ind'
    ↪ definido antes
#           como etiquetas de fila (2,3,5,23,26) en lugar de generar un índice
    ↪ numérico automático.
# Línea 3: muestra la tabla.
# Resultado esperado: tabla de 5 filas x 2 columnas (Name, Age), con índices
    ↪ 2,3,5,23,26.
```

	Name	Age
2	Carlos	34
3	Pedro	22
5	Manolo	15
23	Luis	55
26	Alberto	23

```
[59]: # Son inmutables! No se modifican los datos.
ind[3] = 8
```

```
# Línea 1: comentario descriptivo advirtiendo que los objetos Index son
↳ inmutables.
# Línea 2: intentar modificar un elemento del Index directamente por asignación
↳ lanza un error,
#     ya que los objetos Index de pandas no permiten mutación in place (a
↳ diferencia de
#     listas o ndarrays normales).
# Resultado esperado: TypeError: Index does not support mutable operations
↳ (excepción, no un
#     resultado numérico).
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[59], line 2
      1 # Son inmutables! No se modifican los datos.
----> 2 ind[3] = 8
      3
      4 # Línea 1: comentario descriptivo advirtiendo que los objetos Index son
↳ inmutables.
      5 # Línea 2: intentar modificar un elemento del Index directamente por
↳ asignación lanza un error,

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\pandas\core\indexes\base.py:5382, in Index._setitem__(self, key, value)
    5380 @final
    5381 def _setitem__(self, key, value) -> None:
-> 5382     raise TypeError("Index does not support mutable operations")

TypeError: Index does not support mutable operations
```

```
[60]: # change index column
frame = pd.DataFrame({"Name" : ['Carlos','Pedro', 'Manolo', 'Luis', 'Alberto'],
↳ "Age" : [34,22,15,55,23]}, index=ind)
display(frame)
frame.set_index('Age', inplace=True)
display(frame)

# Línea 1: comentario descriptivo.
# Línea 2: recrea el mismo DataFrame de 5 filas con el índice 'ind'.
# Línea 3: lo muestra.
# Línea 4: set_index('Age', inplace=True) convierte la columna 'Age' en el
↳ nuevo índice del
#     DataFrame (deja de ser una columna normal), modificando 'frame'
↳ directamente.
# Línea 5: muestra el DataFrame con 'Age' como índice.
```

```
# Resultado esperado: primero la tabla original (índice 2,3,5,23,26; columnas
↳Name,Age), luego
# la misma tabla pero con 'Age' (34,22,15,55,23) como índice y solo la
↳columna 'Name'.
```

	Name	Age
2	Carlos	34
3	Pedro	22
5	Manolo	15
23	Luis	55
26	Alberto	23

	Name
34	Carlos
22	Pedro
15	Manolo
55	Luis
23	Alberto

1.5.6 4.5 Slicing

```
[61]: # slice por filas
d_and_d_characters = {'Name' : ['bundenth','theorin','barlok'], 'Strength' :
↳[10,12,19], 'Wisdom' : [20,13,6]}
character_data = pd.DataFrame(d_and_d_characters, index=['a','b','c'])
display(character_data)
display(character_data[:-1])
display(character_data[1:2])

# Línea 1: comentario descriptivo.
# Línea 2: define un diccionario con datos de 3 personajes.
# Línea 3: crea el DataFrame con índices 'a','b','c'.
# Línea 4: muestra la tabla completa.
# Línea 5: indexar un DataFrame directamente con un slice (sin loc/iloc) actúa
↳sobre las FILAS
# por posición;[:-1] toma todas las filas menos la última.
# Línea 6: [1:2] toma solo la fila en la posición 1 (la segunda fila,
↳'theorin').
# Resultado esperado: tabla completa de 3 filas, luego una tabla de 2 filas
↳('a','b'), y luego
# una tabla de 1 fila ('b', theorin).
```

	Name	Strength	Wisdom
a	bundenth	10	20
b	theorin	12	13
c	barlok	19	6

	Name	Strength	Wisdom
	b	12	13

a	bundenth	10	20
b	theorin	12	13

	Name	Strength	Wisdom
b	theorin	12	13

```
[62]: # slicing para columnas
display(character_data[['Name', 'Wisdom']])

# Línea 1: comentario descriptivo.
# Línea 2: indexar con una lista de nombres de columna selecciona solo esas
↳ columnas para
# todas las filas.
# Resultado esperado: tabla de 3 filas x 2 columnas (Name, Wisdom).
```

	Name	Wisdom
a	bundenth	20
b	theorin	13
c	barlok	6

```
[63]: #slicing con 'loc' e 'iloc'
display(character_data.iloc[1:])
display(character_data.loc[:'b', 'Name':'Strength'])

# Línea 1: comentario descriptivo.
# Línea 2: .iloc[1:] selecciona todas las filas desde la posición 1 en adelante
↳ (excluye la
# primera fila, 'a').
# Línea 3: .loc[:'b', 'Name':'Strength'] selecciona las filas hasta 'b'
↳ inclusive, y las
# columnas entre 'Name' y 'Strength' inclusive.
# Resultado esperado: tabla con las filas 'b' y 'c'; y tabla con filas 'a', 'b'
↳ y columnas
# 'Name', 'Strength'.
```

	Name	Strength	Wisdom
b	theorin	12	13
c	barlok	19	6

	Name	Strength
a	bundenth	10
b	theorin	12

¿Cómo filtrar filas y columnas? Por ejemplo, para todos los personajes, obtener ‘Name’ y ‘Strength’

```
[64]: # usando 'loc' para hacer slicing
display(character_data.loc[:, 'Name':'Strength'])

# Línea 1: comentario descriptivo.
```

```
# Línea 2: .loc[:, 'Name':'Strength'] selecciona TODAS las filas y solo las
↳ columnas entre
# 'Name' y 'Strength' inclusive.
# Resultado esperado: tabla de 3 filas x 2 columnas (Name, Strength).
```

	Name	Strength
a	bundenth	10
b	theorin	12
c	barlok	19

```
[65]: # usando 'loc' para buscar específicamente filas y columnas
display(character_data.loc[ ['a','c'], ['Name','Wisdom'] ])

# Línea 1: comentario descriptivo.
# Línea 2: pasar dos listas explícitas a .loc selecciona el producto cruzado de
↳ esas filas
# ('a','c') y columnas ('Name','Wisdom').
# Resultado esperado: tabla de 2 filas ('a','c') x 2 columnas (Name, Wisdom).
```

	Name	Wisdom
a	bundenth	20
c	barlok	6

```
[66]: # lo mismo con 'iloc'?
display(character_data.iloc[[0,2],[0,2]])
display(character_data.iloc[[0,-1],[0,-1]])

# Línea 1: comentario descriptivo (pregunta retórica).
# Línea 2: .iloc[[0,2],[0,2]] selecciona por posición las filas 0 y 2, y las
↳ columnas 0 y 2
# (equivalente posicional al ejemplo anterior con loc).
# Línea 3: .iloc[[0,-1],[0,-1]] usa índices negativos: fila 0 y última fila
↳ (-1), columna 0 y
# última columna (-1), una forma compacta de tomar "primero y último".
# Resultado esperado: dos tablas de 2x2, con las filas/columnas 'Name' y
↳ 'Wisdom' para los
# personajes en las posiciones 0 y 2 (bundenth y barlok) en ambos
↳ casos.
```

	Name	Wisdom
a	bundenth	20
c	barlok	6

	Name	Wisdom
a	bundenth	20
c	barlok	6


```
[67]: # lista de los personajes con el atributo Strength > 11
display(character_data.loc[character_data['Strength'] > 11, ['Name',
↳ 'Strength']])

# Línea 1: comentario descriptivo.
# Línea 2: character_data['Strength'] > 11 genera una máscara booleana por fila;
↳ .loc la usa
#
para filtrar filas, y además se especifica una lista de columnas
↳ ('Name', 'Strength')
#
a mostrar, combinando filtrado de filas y selección de columnas en
↳ un solo paso.
# Resultado esperado: tabla con 'theorin' (Strength=12) y 'barlok'
↳ (Strength=19), mostrando
#
solo sus columnas Name y Strength.
```

	Name	Strength
b	theorin	12
c	barlok	19

```
[68]: # listar los personajes con Strength > 15 o Wisdom > 15
display(character_data.loc[(character_data['Strength'] > 15) |
↳ (character_data['Wisdom'] > 15)])

# Línea 1: comentario descriptivo.
# Línea 2: combina dos condiciones booleanas con el operador OR a nivel de
↳ elemento '|' (cada
#
condición debe ir entre paréntesis); .loc filtra las filas donde al
↳ menos una de las
#
dos condiciones es verdadera.
# Resultado esperado: tabla con 'bundenth' (Wisdom=20>15) y 'barlok'
↳ (Strength=19>15), ambas
#
con todas sus columnas.
```

	Name	Strength	Wisdom
a	bundenth	10	20
c	barlok	19	6

1.6 5. Cargar y guardar datos en pandas

```
[69]: # Guardar a csv
import os
ruta = os.path.join("res" , "o_d_d_characters.csv")
character_data.to_csv(ruta, sep=';') # sep por defecto: ','

# Línea 1: comentario descriptivo.
# Línea 2: importa el módulo os.
# Línea 3: construye la ruta 'res/o_d_d_characters.csv'.
```

```
# Línea 4: (descomentado) to_csv exporta el DataFrame a un archivo CSV usando ';'
↳ ' como
#           separador de columnas en lugar de la coma por defecto (útil, por
↳ ejemplo, en
#           configuraciones regionales donde la coma se usa como separador
↳ decimal).
# Resultado esperado: no hay salida impresa; se crea (o sobrescribe) el archivo
#           'res/o_d_d_characters.csv' en disco con los datos de character_data
↳ separados por ';'.

```

```
[70]: loaded = pd.read_csv(ruta, sep=';')
display(loaded)

# Línea 1: pd.read_csv lee el archivo recién guardado usando el mismo separador
↳ ';'.
# Línea 2: muestra el DataFrame cargado.
# Resultado esperado: una tabla con las mismas filas y columnas que
↳ character_data, pero con
#           una columna adicional 'Unnamed: 0' que contiene el índice original
↳ (a,b,c), ya que
#           to_csv guarda el índice por defecto y read_csv no lo reconoce
↳ automáticamente como
#           índice a menos que se indique index_col.

```

	Unnamed: 0	Name	Strength	Wisdom
0	a	bundenth	10	20
1	b	theorin	12	13
2	c	barlok	19	6

```
[71]: ruta = os.path.join("res","titanic.csv")

# Línea 1: construye la ruta al archivo 'res/titanic.csv', un dataset clásico
↳ de ejemplo.
# Resultado esperado: no hay salida impresa; la ruta queda almacenada para la
↳ siguiente celda.

```

```
[72]: titanic = pd.read_csv(ruta, sep=',')
display(titanic)

# Línea 1: lee el CSV del Titanic usando coma como separador (el formato
↳ estándar del archivo).
# Línea 2: muestra el DataFrame completo.
# Resultado esperado: una tabla con los pasajeros del Titanic y sus atributos
↳ (columnas típicas
#           como PassengerId, Survived, Pclass, Name, Sex, Age, etc., según el
↳ dataset usado).

```

PassengerId	Survived	Pclass	\
-------------	----------	--------	---

0	1	0	3
1	2	1	1
2	3	1	3
3	4	1	1
4	5	0	3
..	...	...	...
886	887	0	2
887	888	1	1
888	889	0	3
889	890	1	1
890	891	0	3

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	
..	...	...	...	...	
886	Montvila, Rev. Juozas	male	27.0	0	
887	Graham, Miss. Margaret Edith	female	19.0	0	
888	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	
889	Behr, Mr. Karl Howell	male	26.0	0	
890	Dooley, Mr. Patrick	male	32.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S
..	...	...	...	...	
886	0	211536	13.0000	NaN	S
887	0	112053	30.0000	B42	S
888	2	W./C. 6607	23.4500	NaN	S
889	0	111369	30.0000	C148	C
890	0	370376	7.7500	NaN	Q

[891 rows x 12 columns]

```
[73]: loaded = pd.read_csv(ruta, sep=',', index_col = 0)
display(loaded)

# Línea 1: index_col=0 indica a pandas que use la PRIMERA columna del CSV como
↳ índice del
# DataFrame, en lugar de crear un índice numérico adicional.
# Línea 2: muestra el DataFrame resultante.
```

```
# Resultado esperado: la misma tabla del Titanic, pero usando la primera
  ↳columna original
#      (normalmente el identificador del pasajero) como índice de fila en
  ↳lugar de columna.
```

PassengerId	Survived	Pclass	\
1	0	3	
2	1	1	
3	1	3	
4	1	1	
5	0	3	
...	...	...	
887	0	2	
888	1	1	
889	0	3	
890	1	1	
891	0	3	

PassengerId	Name	Sex	Age	\
1	Braund, Mr. Owen Harris	male	22.0	
2	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	
3	Heikkinen, Miss. Laina	female	26.0	
4	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	
5	Allen, Mr. William Henry	male	35.0	
...	...	...	...	
887	Montvila, Rev. Juozas	male	27.0	
888	Graham, Miss. Margaret Edith	female	19.0	
889	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	
890	Behr, Mr. Karl Howell	male	26.0	
891	Dooley, Mr. Patrick	male	32.0	

PassengerId	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	1	0	A/5 21171	7.2500	NaN	S
2	1	0	PC 17599	71.2833	C85	C
3	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	0	113803	53.1000	C123	S
5	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...
887	0	0	211536	13.0000	NaN	S
888	0	0	112053	30.0000	B42	S
889	1	2	W./C. 6607	23.4500	NaN	S
890	0	0	111369	30.0000	C148	C
891	0	0	370376	7.7500	NaN	Q

[891 rows x 11 columns]

Otros argumentos de `to_csv()`:

- `na_rep='string'` → representar valores NaN en el archivo csv.

Otros argumentos de `read_csv()`:

- `na_values='string'`

Pandas también ofrece funciones para leer/guardar en otros formatos estándar: JSON, HDF5 o Excel en su API.

1.7 6. Ejemplo práctico en pandas

- Dataset MovieLens.
- Reviews de películas.
- 1 millón de entradas.
- Datos demográficos de usuarios.

```
[74]: import numpy as np
import pandas as pd
import zipfile # para descomprimir archivos zip
import urllib.request # para descargar de URL
import os

# descargar MovieLens dataset
url = 'http://files.grouplens.org/datasets/movielens/ml-1m.zip'
ruta = os.path.join("res", "ml-1m.zip")
urllib.request.urlretrieve(url, ruta)

# Línea 1-5: importa numpy, pandas, zipfile (para descomprimir), urllib.request
# ↳ (para
# ↳ descargar archivos por HTTP) y os (para rutas).
# Línea 6: comentario descriptivo.
# Línea 7: define la URL del dataset MovieLens de 1 millón de valoraciones.
# Línea 8: construye la ruta local de destino 'res/ml-1m.zip'.
# Línea 9: urlretrieve descarga el archivo remoto y lo guarda en la ruta local
# ↳ indicada.
# Resultado esperado: no hay salida impresa directamente (urlretrieve devuelve
# ↳ una tupla que no
# ↳ se captura); se crea el archivo 'res/ml-1m.zip' en disco tras la
# ↳ descarga.
```

```
[74]: ('res\\ml-1m.zip', <http.client.HTTPMessage at 0x225ea8cf590>)
```

```
[75]: # descomprimiendo archivo zip
ruta_ext = os.path.join("res")
with zipfile.ZipFile(ruta, 'r') as z:
    print('Extracting all files...')
    z.extractall(ruta_ext) # destino
    print('Done!')
```

```
# take a look at readme y revisar formatos

# Línea 1: comentario descriptivo.
# Línea 2: define la carpeta destino de extracción ('res').
# Línea 3: abre el archivo zip descargado en modo lectura usando un context
↳manager.
# Línea 4: imprime un mensaje indicando que comienza la extracción.
# Línea 5: extractall descomprime TODOS los archivos del zip dentro de la
↳carpeta 'res'.
# Línea 6: imprime un mensaje de confirmación al terminar.
# Línea 7: comentario recordando revisar el archivo README del dataset para
↳entender los formatos.
# Resultado esperado:
# Extracting all files...
# Done!
# y se crea la carpeta 'res/ml-1m' con los archivos users.dat, movies.dat,
↳ratings.dat, README, etc.
```

Extracting all files...

Done!

```
[76]: ruta_users = os.path.join("res", "ml-1m", "users.dat")
users_dataset = pd.read_csv(ruta_users, sep='::', index_col=0, engine='python')
display(users_dataset)
```

```
# Línea 1: construye la ruta al archivo 'users.dat' dentro de la carpeta
↳extraída.
# Línea 2: lee el archivo usando '::' como separador (formato propio de
↳MovieLens), usa la
# primera columna como índice, y engine='python' porque el separador
↳de más de un
# carácter requiere el motor de parseo en Python puro (el motor en C
↳no lo soporta).
# Línea 3: muestra el DataFrame cargado.
# Resultado esperado: una tabla con los datos de usuarios, pero con problemas
↳de formato ya que
# el archivo no tiene fila de cabecera (el primer usuario se
↳interpreta erróneamente
# como nombres de columna).
```

	F	1.1	10	48067
1				
2	M	56	16	70072
3	M	25	15	55117
4	M	45	7	02460
5	M	25	20	55455
6	F	50	9	55117

```

... .. ... ..
6036 F 25 15 32603
6037 F 45 1 76006
6038 F 56 1 14706
6039 F 45 0 01060
6040 M 25 6 11106

```

[6039 rows x 4 columns]

```

[77]: # Varios problemas
# sin cabecera! primer valor se ha perdido
# las columnas no tienen nombres
pd.read_csv?

# Línea 1-3: comentarios señalando los problemas detectados: el archivo no
# tiene fila de
# cabecera, por lo que pandas toma la primera fila de datos como
# nombres de columna
# (perdiendo ese primer registro), y las columnas terminan con nombres
# poco descriptivos.
# Línea 4: pd.read_csv? es sintaxis especial de IPython/Jupyter que abre la
# documentación
# (docstring) de la función read_csv en un panel de ayuda, para
# revisar qué parámetros
# existen para solucionar el problema (como header y names).
# Resultado esperado: se despliega la ayuda/documentación de pd.read_csv en el
# panel inferior
# de Jupyter; no se genera una tabla.

```

Signature:

```

pd.read_csv(
    filepath_or_buffer: 'FilePath | ReadCsvBuffer[bytes] | ReadCsvBuffer[str]',
    *,
    sep: 'str | None | lib.NoDefault' = <no_default>,
    delimiter: 'str | None | lib.NoDefault' = None,
    header: "int | Sequence[int] | None | Literal['infer']" = 'infer',
    names: 'Sequence[Hashable] | None | lib.NoDefault' = <no_default>,
    index_col: 'IndexLabel | Literal[False] | None' = None,
    usecols: 'UsecolsArgType' = None,
    dtype: 'DtypeArg | None' = None,
    engine: 'CSVEngine | None' = None,
    converters: 'Mapping[HashableT, Callable] | None' = None,
    true_values: 'list | None' = None,
    false_values: 'list | None' = None,
    skipinitialspace: 'bool' = False,
    skiprows: 'list[int] | int | Callable[[Hashable], bool] | None' = None,
    skipfooter: 'int' = 0,
    nrows: 'int | None' = None,

```

```

na_values: 'Hashable | Iterable[Hashable] | Mapping[Hashable, Union[
↳ Iterable[Hashable]] | None' = None,
keep_default_na: 'bool' = True,
na_filter: 'bool' = True,
skip_blank_lines: 'bool' = True,
parse_dates: 'bool | Sequence[Hashable] | None' = None,
date_format: 'str | dict[Hashable, str] | None' = None,
dayfirst: 'bool' = False,
cache_dates: 'bool' = True,
iterator: 'bool' = False,
chunksize: 'int | None' = None,
compression: 'CompressionOptions' = 'infer',
thousands: 'str | None' = None,
decimal: 'str' = '.',
lineterminator: 'str | None' = None,
quotechar: 'str' = '"',
quoting: 'int' = 0,
doublequote: 'bool' = True,
escapechar: 'str | None' = None,
comment: 'str | None' = None,
encoding: 'str | None' = None,
encoding_errors: 'str | None' = 'strict',
dialect: 'str | csv.Dialect | None' = None,
on_bad_lines: 'str' = 'error',
low_memory: 'bool' = True,
memory_map: 'bool' = False,
float_precision: "Literal['high', 'legacy', 'round_trip'] | None" = None,
storage_options: 'StorageOptions | None' = None,
dtype_backend: 'DtypeBackend | lib.NoDefault' = <no_default>,
) -> 'DataFrame | TextFileReader'

```

Docstring:

Read a comma-separated values (csv) file into DataFrame.

Also supports optionally iterating or breaking of the file into chunks.

Additional help can be found in the online docs for
 IO Tools <https://pandas.pydata.org/pandas-docs/stable/user_guide/io.html>`_.

Parameters

`filepath_or_buffer` : str, path object or file-like object

Any valid string path is acceptable. The string could be a URL. Valid URL schemes include http, ftp, s3, gs, and file. For file URLs, a host is expected. A local file could be: `file://localhost/path/to/table.csv`.

If you want to pass in a path object, pandas accepts any `os.PathLike`.

By file-like object, we refer to objects with a `read()` method, such as a file handle (e.g. via builtin `open` function) or `StringIO`.

`sep` : str, default `','`
 Character or regex pattern to treat as the delimiter. If `sep=None`, the C engine cannot automatically detect the separator, but the Python parsing engine can, meaning the latter will be used and automatically detect the separator from only the first valid row of the file by Python's builtin sniffer tool, `csv.Sniffer`.
 In addition, separators longer than 1 character and different from `'\s+'` will be interpreted as regular expressions and will also force the use of the Python parsing engine. Note that regex delimiters are prone to ignoring quoted data. Regex example: `'\r\t'`.

`delimiter` : str, optional
 Alias for `sep`.

`header` : int, Sequence of int, 'infer' or None, default 'infer'
 Row number(s) containing column labels and marking the start of the data (zero-indexed). Default behavior is to infer the column names: if no `names` are passed the behavior is identical to `header=0` and column names are inferred from the first line of the file, if column names are passed explicitly to `names` then the behavior is identical to `header=None`. Explicitly pass `header=0` to be able to replace existing names. The header can be a list of integers that specify row locations for a :class:`~pandas.MultiIndex` on the columns e.g. `[0, 1, 3]`. Intervening rows that are not specified will be skipped (e.g. 2 in this example is skipped). Note that this parameter ignores commented lines and empty lines if `skip_blank_lines=True`, so `header=0` denotes the first line of data rather than the first line of the file.

When inferred from the file contents, headers are kept distinct from each other by renaming duplicate names with a numeric suffix of the form `"{count}"` starting from 1, e.g. `"foo"` and `"foo.1"`.
 Empty headers are named `"Unnamed: {i}"` or `"Unnamed: {i}_level_{level}"` in the case of MultiIndex columns.

`names` : Sequence of Hashable, optional
 Sequence of column labels to apply. If the file contains a header row, then you should explicitly pass `header=0` to override the column names. Duplicates in this list are not allowed.

`index_col` : Hashable, Sequence of Hashable or False, optional
 Column(s) to use as row label(s), denoted either by column labels or column indices. If a sequence of labels or indices is given, :class:`~pandas.MultiIndex` will be formed for the row labels.

Note: `index_col=False` can be used to force pandas to *not* use the first

column as the index, e.g., when you have a malformed file with delimiters at the end of each line.

`usecols` : Sequence of Hashable or Callable, optional
 Subset of columns to select, denoted either by column labels or column indices.
 If list-like, all elements must either be positional (i.e. integer indices into the document columns) or strings that correspond to column names provided either by the user in `names` or inferred from the document header row(s).
 If `names` are given, the document header row(s) are not taken into account. For example, a valid list-like `usecols` parameter would be `[0, 1, 2]` or `['foo', 'bar', 'baz']`. Element order is ignored, so `usecols=[0, 1]` is the same as `[1, 0]`. To instantiate a `pandas.DataFrame` from `data` with element order preserved use `pd.read_csv(data, usecols=['foo', 'bar'])[['foo', 'bar']]` for columns in `['foo', 'bar']` order or `pd.read_csv(data, usecols=['foo', 'bar'])[['bar', 'foo']]` for `['bar', 'foo']` order.

If callable, the callable function will be evaluated against the column names, returning names where the callable function evaluates to `True`. An example of a valid callable argument would be `lambda x: x.upper()` in `['AAA', 'BBB', 'DDD']`. Using this parameter results in much faster parsing time and lower memory usage.

`dtype` : dtype or dict of {Hashable : dtype}, optional
 Data type(s) to apply to either the whole dataset or individual columns. E.g., `{a: np.float64, b: np.int32, c: 'Int64'}`
 Use `str` or `object` together with suitable `na_values` settings to preserve and not interpret `dtype`.
 If `converters` are specified, they will be applied INSTEAD of `dtype` conversion. Specify a `defaultdict` as input where the default determines the `dtype` of the columns which are not explicitly listed.

`engine` : {'c', 'python', 'pyarrow'}, optional
 Parser engine to use. The C and pyarrow engines are faster, while the python engine is currently more feature-complete. Multithreading is currently only supported by the pyarrow engine. Some features of the "pyarrow" engine are unsupported or may not work correctly.

`converters` : dict of {Hashable : Callable}, optional
 Functions for converting values in specified columns. Keys can either be column labels or column indices.

`true_values` : list, optional
 Values to consider as `True` in addition to case-insensitive variants of 'True'.

`false_values` : list, optional

Values to consider as ``False`` in addition to case-insensitive variants of 'False'.

`skipinitialspace` : bool, default False
Skip spaces after delimiter.

`skiprows` : int, list of int or Callable, optional
Line numbers to skip (0-indexed) or number of lines to skip (``int``) at the start of the file.

If callable, the callable function will be evaluated against the row indices, returning ``True`` if the row should be skipped and ``False`` otherwise.

An example of a valid callable argument would be ``lambda x: x in [0, 2]``.

`skipfooter` : int, default 0
Number of lines at bottom of file to skip (Unsupported with ``engine='c'``).

`nrows` : int, optional
Number of rows of file to read. Useful for reading pieces of large files. Refers to the number of data rows in the returned DataFrame, excluding:

- * The header row containing column names.
- * Rows before the header row, if ``header=1`` or larger.

Example usage:

- * To read the first 999,999 (non-header) rows:
``read\_csv(..., nrows=999999)``
- * To read rows 1,000,000 through 1,999,999:
``read\_csv(..., skiprows=1000000, nrows=999999)``

`na_values` : Hashable, Iterable of Hashable or dict of {Hashable : Iterable}, optional
Additional strings to recognize as ``NA``/``NaN``. If ``dict`` passed, specific per-column ``NA`` values. By default the following values are interpreted as
``NaN``: empty string, "NaN", "N/A", "NULL", and other common representations of missing data.

`keep_default_na` : bool, default True
Whether or not to include the default ``NaN`` values when parsing the data. Depending on whether ``na\_values`` is passed in, the behavior is as follows:

- * If ``keep\_default\_na`` is ``True``, and ``na\_values`` are specified, ``na\_values`` is appended to the default ``NaN`` values used for parsing.
- * If ``keep\_default\_na`` is ``True``, and ``na\_values`` are not specified,
only the default ``NaN`` values are used for parsing.

- * If ``keep\_default\_na`` is ``False``, and ``na\_values`` are specified, only the ``NaN`` values specified ``na\_values`` are used for parsing.
- * If ``keep\_default\_na`` is ``False``, and ``na\_values`` are not specified,
no strings will be parsed as ``NaN``.

Note that if ``na\_filter`` is passed in as ``False``, the ``keep\_default\_na`` and ``na\_values`` parameters will be ignored.

na_filter : bool, default True
Detect missing value markers (empty strings and the value of ``na\_values``).
In data without any ``NA`` values, passing ``na\_filter=False`` can improve the performance of reading a large file.

skip_blank_lines : bool, default True
If ``True``, skip over blank lines rather than interpreting as ``NaN`` values.

parse_dates : bool, None, list of Hashable, default None
The behavior is as follows:

- * ``bool``. If ``True`` -> try parsing the index.
- * ``None``. Behaves like ``True`` if ``date\_format`` is specified.
- * ``list`` of ``int`` or names.
e.g. If ``[1, 2, 3]`` -> try parsing columns 1, 2, 3 each as a separate date column.

If a column or index cannot be represented as an array of ``datetime``, say because of an unparsable value or a mixture of timezones, the column or index will be returned unaltered as an ``object`` data type. For non-standard ``datetime`` parsing, use :func:`~pandas.to\_datetime` after :func:`~pandas.read\_csv`.

Note: A fast-path exists for iso8601-formatted dates.

date_format : str or dict of column -> format, optional
Format to use for parsing dates and/or times when used in conjunction with ``parse\_dates``.
The strftime to parse time, e.g. :const:`~"%d/%m/%Y"`. See `strftime documentation`_ <<https://docs.python.org/3/library/datetime.html#strftime-and-strptime-behavior>>_ for more information on choices, though note that :const:`~"%f"`` will parse all the way up to nanoseconds.
You can also pass:

- "ISO8601", to parse any `ISO8601` <https://en.wikipedia.org/wiki/ISO_8601>_ time string (not necessarily in exactly the same format);
- "mixed", to infer the format for each element individually. This is risky, and you should probably use it along with `dayfirst`.

```

.. versionadded:: 2.0.0
dayfirst : bool, default False
    DD/MM format dates, international and European format.
cache_dates : bool, default True
    If ``True``, use a cache of unique, converted dates to apply the ``datetime``
    conversion. May produce significant speed-up when parsing duplicate
    date strings, especially ones with timezone offsets.

iterator : bool, default False
    Return ``TextFileReader`` object for iteration or getting chunks with
    ``get_chunk()``.
chunksize : int, optional
    Number of lines to read from the file per chunk. Passing a value will cause
    the
    function to return a ``TextFileReader`` object for iteration.
    See the `IO Tools docs
    <https://pandas.pydata.org/pandas-docs/stable/io.html#io-chunking>`_
    for more information on ``iterator`` and ``chunksize``.

compression : str or dict, default 'infer'
    For on-the-fly decompression of on-disk data.
    If 'infer' and 'filepath_or_buffer' is
    path-like, then detect compression from the following extensions: '.gz',
    '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
    (otherwise no compression).
    If using 'zip' or 'tar', the ZIP file must contain only
    one data file to be read in.
    Set to ``None`` for no decompression.
    Can also be a dict with key ``'method'`` set
    to one of {``'zip'``, ``'gzip'``, ``'bz2'``,
    ``'zstd'``, ``'xz'``, ``'tar'``} and
    other key-value pairs are forwarded to
    ``zipfile.ZipFile``, ``gzip.GzipFile``,
    ``bz2.BZ2File``, ``zstandard.ZstdDecompressor``, ``lzma.LZMAFile`` or
    ``tarfile.TarFile``, respectively.
    As an example, the following could be passed for
    Zstandard decompression using a
    custom compression dictionary:
    ``compression={'method': 'zstd', 'dict_data': my_compression_dict}``.

thousands : str (length 1), optional
    Character acting as the thousands separator in numerical values.
decimal : str (length 1), default '.'
    Character to recognize as decimal point (e.g., use ',' for European data).
lineterminator : str (length 1), optional
    Character used to denote a line break. Only valid with C parser.

```

`quotechar` : str (length 1), optional
 Character used to denote the start and end of a quoted item. Quoted items can include the ```delimiter``` and it will be ignored.

`quoting` : {0 or `csv.QUOTE_MINIMAL`, 1 or `csv.QUOTE_ALL`, 2 or `csv.QUOTE_NONNUMERIC`, 3 or `csv.QUOTE_NONE`}, default `csv.QUOTE_MINIMAL`
 Control field quoting behavior per ```csv.QUOTE_*``` constants. Default is ```csv.QUOTE_MINIMAL``` (i.e., 0) which implies that only fields containing special characters are quoted (e.g., characters defined in ```quotechar```, ```delimiter```, or ```lineterminator```).

`doublequote` : bool, default True
 When ```quotechar``` is specified and ```quoting``` is not ```QUOTE_NONE```,
 indicate whether or not to interpret two consecutive ```quotechar``` elements INSIDE a field as a single ```quotechar``` element.

`escapechar` : str (length 1), optional
 Character used to escape other characters.

`comment` : str (length 1), optional
 Character indicating that the remainder of line should not be parsed. If found at the beginning of a line, the line will be ignored altogether. This parameter must be a single character. Like empty lines (as long as ```skip_blank_lines=True```), fully commented lines are ignored by the parameter ```header``` but not by ```skiprows```. For example, if ```comment='#'```, parsing ```#empty\na,b,c\n1,2,3``` with ```header=0``` will result in ```'a,b,c'``` being treated as the header.

`encoding` : str, optional, default 'utf-8'
 Encoding to use for UTF when reading/writing (ex. ```'utf-8'```). List of
 Python standard encodings
<https://docs.python.org/3/library/codecs.html#standard-encodings>>`_ .

`encoding_errors` : str, optional, default 'strict'
 How encoding errors are treated. List of possible values
<https://docs.python.org/3/library/codecs.html#error-handlers>>`_ .

`dialect` : str or `csv.Dialect`, optional
 If provided, this parameter will override values (default or not) for the following parameters: ```delimiter```, ```doublequote```, ```escapechar```, ```skipinitialspace```, ```quotechar```, and ```quoting```. If it is necessary to override values, a ```ParserWarning``` will be issued. See ```csv.Dialect``` documentation for more details.

`on_bad_lines` : {'error', 'warn', 'skip'} or Callable, default 'error'
 Specifies what to do upon encountering a bad line (a line with too many
 fields).
 Allowed values are:

- ``'error'``, raise an Exception when a bad line is encountered.
- ``'warn'``, raise a warning when a bad line is encountered and skip that line.
- ``'skip'``, skip bad lines without raising or warning when they are encountered.
- Callable, function that will process a single bad line.
 - With ``engine='python'``, function with signature `` (bad\_line: list[str]) -> list[str] | None ``. ``bad\_line`` is a list of strings split by the ``sep``. If the function returns ``None``, the bad line will be ignored. If the function returns a new ``list`` of strings with more elements than expected, a ``ParserWarning`` will be emitted while dropping extra elements.
 - With ``engine='pyarrow'``, function with signature as described in pyarrow documentation: `invalid_row_handler` <https://arrow.apache.org/docs/python/generated/pyarrow.csv.ParseOptions.html#pyarrow.csv.ParseOptions.invalid_row_handler>`_.

.. versionchanged:: 2.2.0

Callable for ``engine='pyarrow'``

`low_memory` : bool, default True

Internally process the file in chunks, resulting in lower memory use while parsing, but possibly mixed type inference. To ensure no mixed types either set ``False``, or specify the type with the ``dtype`` parameter. Note that the entire file is read into a single `pandas.DataFrame` regardless, use the ``chunksize`` or ``iterator`` parameter to return the data in chunks. (Only valid with C parser).

`memory_map` : bool, default False

If a filepath is provided for ``filepath\_or\_buffer``, map the file object directly onto memory and access the data directly from there. Using this option can improve performance because there is no longer any I/O overhead.

`float_precision` : {'high', 'legacy', 'round_trip'}, optional

Specifies which converter the C engine should use for floating-point values. The options are ``None`` or ``'high'`` for the ordinary converter, ``'legacy'`` for the original lower precision pandas converter, and ``'round\_trip'`` for the round-trip converter.

`storage_options` : dict, optional

Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are

forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>` `\_`.

```
dtype_backend : {'numpy_nullable', 'pyarrow'}
    Back-end data type applied to the resultant :class:`DataFrame`
    (still experimental). If not specified, the default behavior
    is to not use nullable data types. If specified, the behavior
    is as follows:

    * ``"numpy_nullable"``: returns nullable-dtype-backed :class:`DataFrame`
    * ``"pyarrow"``: returns
      pyarrow-backed nullable :class:`ArrowDtype` :class:`DataFrame`

    .. versionadded:: 2.0
```

Returns

DataFrame or TextFileReader

A comma-separated values (csv) file is returned as two-dimensional data structure with labeled axes.

See Also

DataFrame.to_csv : Write DataFrame to a comma-separated values (csv) file.
read_table : Read general delimited file into DataFrame.
read_fwf : Read a table of fixed-width formatted lines into DataFrame.

Examples

>>> pd.read_csv("data.csv") # doctest: +SKIP
 Name Value
0 foo 1
1 bar 2
2 #baz 3

Index and header can be specified via the `index\_col` and `header` arguments.

```
>>> pd.read_csv("data.csv", header=None) # doctest: +SKIP
   0      1
0  Name  Value
1  foo      1
2  bar      2
3  #baz      3
```

```
>>> pd.read_csv("data.csv", index_col="Value") # doctest: +SKIP
   Name
```


	Value
1	foo
2	bar
3	#baz

Column types are inferred but can be explicitly specified using the `dtype` argument.

```
>>> pd.read_csv("data.csv", dtype={"Value": float}) # doctest: +SKIP
   Name  Value
0  foo    1.0
1  bar    2.0
2 #baz    3.0
```

True, False, and NA values, and thousands separators have defaults, but can be explicitly specified, too. Supply the values you would like as strings or lists of strings!

```
>>> pd.read_csv("data.csv", na_values=["foo", "bar"]) # doctest: +SKIP
   Name  Value
0  NaN      1
1  NaN      2
2 #baz      3
```

Comment lines in the input file can be skipped using the `comment` argument.

```
>>> pd.read_csv("data.csv", comment="#") # doctest: +SKIP
   Name  Value
0  foo      1
1  bar      2
```

By default, columns with dates will be read as `object` rather than `datetime`.

```
>>> df = pd.read_csv("tmp.csv") # doctest: +SKIP
```

```
>>> df # doctest: +SKIP
   col 1      col 2      col 3
0    10  10/04/2018  Sun 15 Jan 2023
1    20  15/04/2018  Fri 12 May 2023
```

```
>>> df.dtypes # doctest: +SKIP
col 1      int64
col 2      object
col 3      object
dtype: object
```

Specific columns can be parsed as dates by using the `parse\_dates` and `date\_format` arguments.

```
>>> df = pd.read_csv(
...     "tmp.csv",
...     parse_dates=[1, 2],
...     date_format={"col 2": "%d/%m/%Y", "col 3": "%a %d %b %Y"},
... ) # doctest: +SKIP

>>> df.dtypes # doctest: +SKIP
col 1      int64
col 2    datetime64[ns]
col 3    datetime64[ns]
dtype: object
File:      c:
↳ \users\user\anaconda3\envs\machinelearning\lib\site-packages\pandas\io\parsers\readers.
↳ py
Type:      function
```

```
[78]: # especificar nombres, cargar sin cabecera
users_dataset = pd.read_csv(ruta_users, sep='::', index_col=0,
                             header=None,
                             names=['UserID', 'Gender', 'Age', 'Occupation', 'Zip-code'],
                             engine='python')
display(users_dataset)

# Línea 1: comentario descriptivo.
# Línea 2-4: se vuelve a leer el archivo, pero ahora con header=None (indicando
↳ que el archivo
#
↳ NO tiene fila de cabecera, así que ninguna fila de datos se pierde)
↳ y names=[...]
#
↳ (asignando manualmente los nombres de columna correctos);
↳ index_col=0 sigue usando
#
↳ la primera columna (UserID) como índice.
# Línea 5: muestra el DataFrame ya corregido.
# Resultado esperado: tabla con columnas Gender, Age, Occupation, Zip-code,
↳ indexada por
#
↳ UserID, y ahora SÍ incluye el primer usuario que antes se perdía.
```

	Gender	Age	Occupation	Zip-code
UserID				
1	F	1	10	48067
2	M	56	16	70072
3	M	25	15	55117
4	M	45	7	02460
5	M	25	20	55455
...	...	...	...	...
6036	F	25	15	32603

6037	F	45	1	76006
6038	F	56	1	14706
6039	F	45	0	01060
6040	M	25	6	11106

[6040 rows x 4 columns]

```
[79]: # samplear la tabla
display(users_dataset.sample(10))

# Línea 1: comentario descriptivo.
# Línea 2: sample(10) devuelve 10 filas seleccionadas AL AZAR del DataFrame
↳ (útil para
# inspeccionar datos sin sesgo de orden).
# Resultado esperado: una tabla de 10 filas elegidas aleatoriamente (distintas
↳ en cada
# ejecución, salvo que se fije una semilla).
```

	Gender	Age	Occupation	Zip-code
UserID				
4802	M	56	1	40601
4901	M	25	7	11228
5134	M	25	17	95014
4109	M	35	3	97202
2632	M	56	0	43528
5211	M	25	1	40214
5145	M	35	7	77565-2332
4491	M	18	4	52405
4036	M	25	1	46530
5724	M	25	15	94102

```
[80]: # samplear la cabeza
display(users_dataset.head(4))

# Línea 1: comentario descriptivo.
# Línea 2: head(4) devuelve las primeras 4 filas del DataFrame en su orden
↳ original.
# Resultado esperado: tabla con los 4 primeros usuarios (UserID 1 a 4).
```

	Gender	Age	Occupation	Zip-code
UserID				
1	F	1	10	48067
2	M	56	16	70072
3	M	25	15	55117
4	M	45	7	02460

```
[81]: # samplear la cola
display(users_dataset.tail(10))
```

```
# Línea 1: comentario descriptivo.
# Línea 2: tail(10) devuelve las últimas 10 filas del DataFrame.
# Resultado esperado: tabla con los 10 últimos usuarios registrados en el
↳ archivo.
```

	Gender	Age	Occupation	Zip-code
UserID				
6031	F	18	0	45123
6032	M	45	7	55108
6033	M	50	13	78232
6034	M	25	14	94117
6035	F	25	1	78734
6036	F	25	15	32603
6037	F	45	1	76006
6038	F	56	1	14706
6039	F	45	0	01060
6040	M	25	6	11106

```
[82]: # tipos de datos sobre las columnas
users_dataset.dtypes

# Línea 1: comentario descriptivo.
# Línea 2: .dtypes muestra el tipo de dato inferido para cada columna del
↳ DataFrame; al ser la
# última expresión de la celda se muestra automáticamente.
# Resultado esperado: una lista tipo Series con el tipo de cada columna, por
↳ ejemplo Gender
# (object), Age (int64), Occupation (int64), Zip-code (object).
```

```
[82]: Gender      str
Age      int64
Occupation  int64
Zip-code   str
dtype: object
```

```
[83]: display(users_dataset[users_dataset['Zip-code'].str.len() > 5])

# Línea 1: .str.len() aplica la función len() a cada valor de la columna
↳ 'Zip-code' (accesor
# .str permite usar funciones de string vectorizadas sobre toda la
↳ columna); la
# condición filtra las filas donde el código postal tiene más de 5
↳ caracteres.
# Resultado esperado: una tabla con los usuarios cuyo Zip-code es más largo de
↳ lo esperado (por
```

```
#          ejemplo, códigos postales de EE.UU. con extensión '+4', como
↳ '12345-6789').
```

	Gender	Age	Occupation	Zip-code
UserID				
161	M	45	16	98107-2117
233	F	45	20	37919-4204
293	M	56	1	55337-4056
458	M	50	16	55405-2546
506	M	25	16	55103-1006
...	...	...	...	...
5682	M	18	0	23455-4959
5904	F	45	12	954025
5925	F	25	0	90035-4444
5967	M	50	16	73069-5429
5985	F	18	4	78705-5221

[81 rows x 4 columns]

```
[84]: # información general sobre atributos numéricos
display(users_dataset.describe())

# Línea 1: comentario descriptivo.
# Línea 2: describe() calcula estadísticas descriptivas (count, mean, std, min,
↳ cuartiles, max)
#          para las columnas NUMÉRICAS del DataFrame por defecto.
# Resultado esperado: tabla con estadísticas para la columna 'Age' y
↳ 'Occupation' (las únicas
#          numéricas), por ejemplo su media, desviación estándar, mínimo y
↳ máximo.
```

	Age	Occupation
count	6040.000000	6040.000000
mean	30.639238	8.146854
std	12.895962	6.329511
min	1.000000	0.000000
25%	25.000000	3.000000
50%	25.000000	7.000000
75%	35.000000	14.000000
max	56.000000	20.000000

```
[85]: users_dataset.info()

# Línea 1: info() imprime un resumen conciso del DataFrame: número de filas,
↳ número de
#          columnas, tipo de dato de cada columna, cantidad de valores no nulos
↳ y uso de memoria.
# Resultado esperado: un bloque de texto tipo:
```

```
# <class 'pandas.core.frame.DataFrame'>
# Index: 6040 entries, 1 to 6040
# Data columns (total 4 columns):
# #   Column      Non-Null Count  Dtype
# ...
# (los valores exactos dependen del dataset MovieLens 1M).
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6040 entries, 1 to 6040
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Gender      6040 non-null   str
1   Age         6040 non-null   int64
2   Occupation  6040 non-null   int64
3   Zip-code    6040 non-null   str
dtypes: int64(2), str(2)
memory usage: 188.9 KB
```

```
[86]: # incluir otros atributos (no todo tiene sentido)
display(users_dataset.describe(include='all'))

# Línea 1: comentario descriptivo.
# Línea 2: describe(include='all') fuerza a incluir también las columnas NO
↳ numéricas (como
#           Gender y Zip-code), calculando estadísticas aplicables a texto
↳ (count, unique, top,
#           freq) además de las numéricas; las celdas no aplicables quedan en
↳ NaN.
# Resultado esperado: tabla ampliada con estadísticas mixtas para todas las
↳ columnas, con NaN
#           en las combinaciones estadístico/columna que no tienen sentido (p.
↳ ej. 'mean' de Gender).
```

	Gender	Age	Occupation	Zip-code
count	6040	6040.000000	6040.000000	6040
unique	2	NaN	NaN	3439
top	M	NaN	NaN	48104
freq	4331	NaN	NaN	19
mean	NaN	30.639238	8.146854	NaN
std	NaN	12.895962	6.329511	NaN
min	NaN	1.000000	0.000000	NaN
25%	NaN	25.000000	3.000000	NaN
50%	NaN	25.000000	7.000000	NaN
75%	NaN	35.000000	14.000000	NaN
max	NaN	56.000000	20.000000	NaN

```
[87]: # cuántos usuarios son mujeres (Gender='F')
len(users_dataset[users_dataset['Gender'] == 'F'])
#select count(*) from users_dataset where users_dataset.Gender = 'F'

# Línea 1: comentario descriptivo.
# Línea 2: filtra las filas donde Gender es 'F' y aplica len() para contar
    ↳ cuántas filas cumplen
#         la condición (equivalente a un COUNT con WHERE en SQL).
# Línea 3: comentario que muestra la consulta SQL equivalente conceptualmente.
# Resultado esperado: un entero con el número total de usuarias mujeres en el
    ↳ dataset.
```

[87]: 1709

```
[88]: # mostrar solo los menores de edad
under_age = users_dataset[users_dataset['Age'] == 1]
print(len(under_age))
display(under_age.sample(10))

# Línea 1: comentario descriptivo.
# Línea 2: filtra los usuarios cuya columna Age tiene el valor codificado 1,
    ↳ que en este
#         dataset (MovieLens) representa la categoría de edad "menores de 18".
# Línea 3: imprime cuántos usuarios cumplen esa condición.
# Línea 4: muestra una muestra aleatoria de 10 de esos usuarios menores de edad.
# Resultado esperado: un número entero de usuarios menores de edad, seguido de
    ↳ una tabla de 10
#         filas aleatorias de ese subconjunto.
```

222

	Gender	Age	Occupation	Zip-code
UserID				
1056	M	1	0	48133
1425	M	1	10	08872
1818	M	1	8	02254
4086	F	1	10	55391
2288	M	1	10	13066
3234	F	1	0	90012
3780	M	1	0	46979
3087	F	1	1	90802
1200	M	1	0	97222
926	M	1	10	07869

```
[89]: # filtrar edad incorrecta (mínimo 18) SettingWithCopyWarning!!!
users_dataset = pd.read_csv(ruta_users, sep='::', index_col=0,
                             header=None,
    ↳ names=['UserID', 'Gender', 'Age', 'Occupation', 'Zip-code'],
```

```

engine='python')
under_age = users_dataset[users_dataset['Age'] == 1]
under_age.loc['Age'] = np.nan
display(under_age.head())
# users_dataset[users_dataset['Age'] < 18] = under_age
# display(users_dataset)

# Línea 1: comentario descriptivo indicando que este intento generará una
↳ advertencia.
# Línea 2-4: se vuelve a cargar el dataset original limpio.
# Línea 5: se filtra un subconjunto (usuarios con Age=1); ESTE SUBCONJUNTO es
↳ potencialmente
# una VISTA (no una copia garantizada) del DataFrame original.
# Línea 6: under_age.loc['Age'] = np.nan es en realidad un ERROR CONCEPTUAL en
↳ el material:
# intenta asignar a una FILA con etiqueta 'Age' (que no existe como
↳ índice), en lugar
# de a la COLUMNA 'Age'; esto añade una fila nueva con índice 'Age' en
↳ vez de modificar
# la columna, y además dispara un SettingWithCopyWarning por trabajar
↳ sobre un
# subconjunto potencialmente vista.
# Línea 7: muestra las primeras filas de 'under_age' ya con el efecto no
↳ deseado.
# Línea 8-9: (quedan comentadas intencionalmente) muestran el intento fallido
↳ de propagar los
# cambios de vuelta al DataFrame original usando una máscara booleana
↳ como asignación
# directa, lo cual no funciona como se espera y por eso se mantiene
↳ comentado.
# Resultado esperado: se emite un SettingWithCopyWarning en consola/salida, y
↳ under_age.head()
# muestra una fila adicional inesperada con índice 'Age', evidenciando
↳ el problema que
# se corrige en las dos celdas siguientes.

```

UserID	Gender	Age	Occupation	Zip-code
1	F	1.0	10.0	48067
19	M	1.0	10.0	48073
51	F	1.0	10.0	10562
75	F	1.0	10.0	01748
86	F	1.0	10.0	54467

```

[90]: # filtrar edad incorrecta (mínimo 18) Fixing it by Copying the slice
users_dataset = pd.read_csv(ruta_users, sep='::', index_col=0,

```



```

                                header=None,
↳names=['UserID', 'Gender', 'Age', 'Occupation', 'Zip-code'],
                                engine='python')
under_age = users_dataset[users_dataset['Age'] == 1]
under_age_copy = under_age.copy()
display(under_age_copy.head())
under_age_copy['Age'] = np.nan
display(under_age_copy.head())
users_dataset[users_dataset['Age'] == 1] = under_age_copy # sets the rows
↳according to the indexes
display(users_dataset.head())

# Línea 1: comentario descriptivo (solución al problema anterior).
# Línea 2-4: se recarga el dataset original limpio.
# Línea 5: filtra de nuevo los usuarios con Age=1.
# Línea 6: .copy() crea una copia EXPLÍCITA e independiente, evitando el
↳problema de la vista.
# Línea 7: muestra las primeras filas de la copia (aún con Age=1).
# Línea 8: ahora sí se asigna correctamente np.nan a la COLUMNA 'Age' de la
↳copia (sin el error
#
de índice de la celda anterior).
# Línea 9: muestra la copia con Age ya en NaN.
# Línea 10: se usa la máscara booleana sobre el DataFrame original como índice
↳de asignación
#
para sobrescribir esas filas específicas con los valores de la copia
↳modificada
#
(pandas alinea por índice, así que cada fila se actualiza
↳correctamente).
# Línea 11: muestra las primeras filas del dataset original ya actualizado.
# Resultado esperado: no hay warning; la copia muestra Age=NaN, y el dataset
↳original refleja
#
ese cambio únicamente en las filas que originalmente tenían Age=1.

```

	Gender	Age	Occupation	Zip-code
UserID				
1	F	1	10	48067
19	M	1	10	48073
51	F	1	10	10562
75	F	1	10	01748
86	F	1	10	54467

	Gender	Age	Occupation	Zip-code
UserID				
1	F	NaN	10	48067
19	M	NaN	10	48073
51	F	NaN	10	10562
75	F	NaN	10	01748
86	F	NaN	10	54467

	Gender	Age	Occupation	Zip-code
UserID				
1	F	NaN	10	48067
2	M	56.0	16	70072
3	M	25.0	15	55117
4	M	45.0	7	02460
5	M	25.0	20	55455

```
[91]: # filtrar edad incorrecta (mínimo 18) Remove them from the dataset
users_dataset = pd.read_csv(ruta_users, sep='::', index_col=0,
                             header=None,
                             names=['UserID', 'Gender', 'Age', 'Occupation', 'Zip-code'],
                             engine='python')
display(users_dataset[users_dataset['Age'] == 1].head(4))
users_dataset.loc[users_dataset['Age'] == 1, 'Age'] = np.nan
display(users_dataset)
display(users_dataset.loc[pd.isnull(users_dataset['Age'])].head(4))
users_dataset.drop(users_dataset[pd.isnull(users_dataset['Age'])].index,
                    inplace = True)
display(users_dataset.head(4))

# Línea 1: comentario descriptivo (alternativa: eliminar directamente las filas
#           problemáticas).
# Línea 2-4: se recarga el dataset original limpio.
# Línea 5: muestra las primeras 4 filas de usuarios con Age=1, solo para
#           verificar antes del cambio.
# Línea 6: forma CORRECTA y recomendada de modificar un subconjunto: usar .
#           loc[condición,
#           'columna'] = valor, que asigna np.nan directamente en la columna
#           'Age' de las filas
#           que cumplen la condición, sin ambigüedad de vista/copia.
# Línea 7: muestra el DataFrame completo con esos valores ya en NaN.
# Línea 8: filtra y muestra las primeras 4 filas donde 'Age' es nulo (para
#           confirmar el cambio).
# Línea 9: drop(..., inplace=True) elimina del DataFrame TODAS las filas cuyo
#           índice está en el
#           subconjunto de filas con Age nulo, modificando 'users_dataset'
#           directamente.
# Línea 10: muestra las primeras 4 filas del dataset ya sin los usuarios
#           menores de edad.
# Resultado esperado: el dataset final ya NO contiene ningún usuario con Age
#           nulo/menor de edad;
#           el número total de filas es menor que el original en la cantidad de
#           menores eliminados.
```

	Gender	Age	Occupation	Zip-code
UserID				

1	F	1	10	48067
19	M	1	10	48073
51	F	1	10	10562
75	F	1	10	01748

	Gender	Age	Occupation	Zip-code
UserID				
1	F	NaN	10	48067
2	M	56.0	16	70072
3	M	25.0	15	55117
4	M	45.0	7	02460
5	M	25.0	20	55455
...	...	...	...	...
6036	F	25.0	15	32603
6037	F	45.0	1	76006
6038	F	56.0	1	14706
6039	F	45.0	0	01060
6040	M	25.0	6	11106

[6040 rows x 4 columns]

	Gender	Age	Occupation	Zip-code
UserID				
1	F	NaN	10	48067
19	M	NaN	10	48073
51	F	NaN	10	10562
75	F	NaN	10	01748

	Gender	Age	Occupation	Zip-code
UserID				
2	M	56.0	16	70072
3	M	25.0	15	55117
4	M	45.0	7	02460
5	M	25.0	20	55455

```
[92]: # Agrupar datos por atributos
display(users_dataset.groupby(by='Gender').describe())

# Línea 1: comentario descriptivo.
# Línea 2: groupby(by='Gender') agrupa las filas según el valor de la columna
↳ 'Gender' ('F'/'M');
# describe() aplicado sobre el objeto agrupado calcula estadísticas
↳ descriptivas
# (count, mean, std, min, cuartiles, max) por SEPARADO para cada grupo.
# Resultado esperado: una tabla con dos filas (F y M) y columnas jerárquicas
↳ mostrando las
# estadísticas de Age y Occupation calculadas independientemente para
↳ cada género.
```

	Age								Occupation \
	count	mean	std	min	25%	50%	75%	max	count
Gender									
F	1631.0	32.287554	11.792015	18.0	25.0	25.0	45.0	56.0	1631.0
M	4187.0	31.568665	11.716053	18.0	25.0	25.0	35.0	56.0	4187.0

	mean	std	min	25%	50%	75%	max
Gender							
F	6.498467	5.960285	0.0	1.0	4.0	11.0	20.0
M	8.743253	6.441753	0.0	4.0	7.0	15.0	20.0

```
[93]: # Grabar la tabla modificada
# Cambiar el separador a ','
# Guardar NaN como 'null'
ruta_output = os.path.join('res', 'ml-1m', 'o_users_processed.csv')
users_dataset.to_csv(ruta_output, sep=',', na_rep='null')

# Línea 1-3: comentarios describiendo el objetivo de la celda.
# Línea 4: construye la ruta de salida 'res/ml-1m/o_users_processed.csv'.
# Línea 5: to_csv guarda el DataFrame ya procesado, usando coma como separador
↳ (formato CSV
# estándar) y representando cualquier valor NaN restante como el texto
↳ 'null' en el
# archivo (na_rep).
# Resultado esperado: no hay salida impresa; se crea el archivo CSV procesado
↳ en disco.
```

1.8 7. Ejercicios

- Hacer un análisis general de los otros dos archivos CSV en ml-1m (movies.dat y ratings.dat).
- Analizando el dataset ratings.dat, ¿hay algún usuario que no tenga ninguna review? ¿Cuántos tienen menos de 30 reviews?

```
[94]: # especificar nombres, cargar sin cabecera
import os
import pandas as pd
ruta_ratings = os.path.join("res", "ml-1m", "ratings.dat")
ratings_dataset = pd.read_csv(ruta_ratings, sep='::',
                              header=None,
                              names=['UserID', 'MovieID', 'Rating', 'Timestamp'],
                              engine='python')
display(ratings_dataset)

# Línea 1: comentario descriptivo.
# Línea 2-3: importa os y pandas (por si el kernel se reinició).
# Línea 4: construye la ruta al archivo 'ratings.dat'.
```

```
# Línea 5-7: lee el archivo con separador ':::', sin cabecera (header=None),
↳ asignando
#           manualmente los nombres de columna UserID, MovieID, Rating,
↳ Timestamp; sin
#           index_col, por lo que pandas usa un índice numérico automático (útil
↳ aquí porque no
#           hay una clave única por fila, ya que cada fila es una valoración
↳ individual).
# Línea 8: muestra el DataFrame completo.
# Resultado esperado: una tabla de aproximadamente 1.000.209 filas x 4 columnas
↳ con las
#           valoraciones de películas por usuario.
```

	UserID	MovieID	Rating	Timestamp
0	1	1193	5	978300760
1	1	661	3	978302109
2	1	914	3	978301968
3	1	3408	4	978300275
4	1	2355	5	978824291
...	...	...	...	...
1000204	6040	1091	1	956716541
1000205	6040	1094	5	956704887
1000206	6040	562	5	956704746
1000207	6040	1096	4	956715648
1000208	6040	1097	4	956715569

[1000209 rows x 4 columns]

```
[95]: # hay algún usuario que no tenga ninguna review
n_users_with_reviews = len(ratings_dataset.groupby(by='UserID'))
print(n_users_with_reviews)
n_users = len(users_dataset)
print(n_users)
print(n_users - n_users_with_reviews)

# Línea 1: comentario descriptivo.
# Línea 2: agrupa las valoraciones por UserID; el número de grupos resultantes
↳ equivale al
#           número de usuarios DISTINTOS que aparecen al menos una vez en
↳ ratings_dataset (es
#           decir, que tienen al menos una review).
# Línea 3: imprime ese número de usuarios con al menos una review.
# Línea 4: obtiene el número total de usuarios registrados en users_dataset
↳ (recordar que este
#           ya fue filtrado/modificado en celdas anteriores, por lo que puede no
↳ coincidir con
#           el total original de 6040).
```

```

# Línea 5: imprime el total de usuarios.
# Línea 6: la diferencia entre ambos números indica cuántos usuarios (del
↳dataset actual de
#
users_dataset) NO tienen ninguna review asociada; si el resultado es
↳0, significa que
#
todos los usuarios listados tienen al menos una valoración.
# Resultado esperado: dos números similares (usuarios con reviews y total de
↳usuarios) y una
#
diferencia que, en el dataset original completo de MovieLens 1M,
↳suele ser 0 (todos
#
los usuarios tienen al menos una review), aunque puede variar si
↳users_dataset fue
#
modificado previamente en el notebook.

```

6040

5818

-222

```

[96]: # Cuántos tienen menos de 30 reviews
ratings_under_30 = ratings_dataset.groupby(['UserID'])['UserID'].filter(lambda
↳x: len(x) < 30).value_counts()
display(ratings_under_30)
print(len(ratings_under_30))

# Línea 1: comentario descriptivo.
# Línea 2: agrupa por UserID; filter aplica una función lambda a cada grupo y
↳CONSERVA solo las
#
filas de los grupos (usuarios) cuyo tamaño (número de reviews) es
↳menor a 30; el
#
resultado sigue siendo una Series de UserID (una entrada por cada
↳review de esos
#
usuarios). Sobre ese resultado filtrado, value_counts() cuenta
↳cuántas veces aparece
#
cada UserID, es decir, cuántas reviews tiene cada uno de esos
↳usuarios "poco activos".
# Línea 3: muestra la Series resultante (UserID -> número de reviews, ordenada
↳de mayor a menor
#
conteo).
# Línea 4: imprime cuántos usuarios distintos cumplen la condición de tener
↳menos de 30 reviews
#
(longitud de la Series resultante, ya que cada UserID aparece una
↳sola vez en el
#
índice de value_counts()).
# Resultado esperado: una Series con el UserID como índice y su número de
↳reviews (siempre <30)

```

```
# como valor, seguida de un entero con el total de usuarios con menos  
↪ de 30 reviews.
```

UserID

71 29

221 29

234 29

246 29

282 29

..

5533 20

5725 20

5828 20

5904 20

6038 20

Name: count, Length: 751, dtype: int64

751